COMPLETE PRACTICE GUIDE

SQL Complete *Guide*

350 Practice Questions & Solutions

"350 real, scenario-based SQL problems with working query solutions — covering joins, aggregation, ranking, window functions, dates, subqueries, recursive CTEs, schema design, performance tuning, and the toughest senior-level interview favorites."

Questions · **350** | Level · **Beginner → Senior** | Topics · **16**

Dialect · **ANSI / PostgreSQL**

- ▸ Filtering & Aggregation
- ▸ Joins & Multi-table Queries
- ▸ Duplicates & Nth-Value Problems
- ▸ Window Functions
- ▸ Date & Time Logic
- ▸ GROUP BY / HAVING Puzzles
- ▸ Subqueries & Recursive CTEs
- ▸ String & Pattern Matching
- ▸ Complex JOIN Patterns
- ▸ Aggregation Pipelines
- ▸ Schema Modifications & DDL
- ▸ Performance Tuning
- ▸ Gaps, Islands & Pivoting
- ▸ Recursive Hierarchies
- ▸ Real Interview Favorites
- ▸ Senior-Level Challenges

Created By: Kartik Mungole

@kartikmungole

Reference: Database Schema

All 350 questions in this guide are written against the sample database below. Keep this page open as a reference — column names used in every query map directly to these tables.

▸ employees

COLUMN	TYPE	DESCRIPTION
id	INT (PK)	Unique employee ID
name	VARCHAR	Employee full name
department_id	INT (FK)	Links to departments.department_id
manager_id	INT (FK, self)	Links to another employee's id
job_title	VARCHAR	Designation, e.g. Manager, Analyst
salary	DECIMAL	Monthly/annual salary
join_date	DATE	Date of hire
email	VARCHAR	Employee email address

▸ departments

COLUMN	TYPE	DESCRIPTION
department_id	INT (PK)	Unique department ID
department_name	VARCHAR	e.g. Engineering, Sales, HR
location	VARCHAR	City/office location

▸ customers

COLUMN	TYPE	DESCRIPTION
customer_id	INT (PK)	Unique customer ID
name	VARCHAR	Customer full name
email	VARCHAR	Customer email
phone	VARCHAR	Contact number
signup_date	DATE	Date customer registered
country	VARCHAR	Customer's country

▸ orders & order_items

TABLE	COLUMN	DESCRIPTION
orders	order_id (PK), customer_id (FK), order_date, amount	One row per order placed
order_items	order_id (FK), product_id (FK), quantity, price	Line items within an order

▸ products & categories

TABLE	COLUMN	DESCRIPTION
products	product_id (PK), product_name, category_id (FK), price, vendor_id, stock_quantity	Product catalog
categories	category_id (PK), category_name	Product category lookup

Note: a few advanced and senior-level questions also reference smaller, self-explanatory tables (e.g. logins, leave_records, salary_history, project_assignments, subscriptions, ledger_a/ledger_b) — their purpose is always clear from the column names used in that specific query.

Reference: Sample Data

Sample rows for the core tables — use these to mentally trace through any query's logic and verify what the output would look like.

▸ employees (sample rows)

ID	NAME	DEPT_ID	MANAGER_ID	JOB_TITLE	SALARY	JOIN_DATE
1	John Smith	10	NULL	Director	120000	2018-03-01
2	Aman Verma	10	1	Manager	85000	2019-06-15
3	Priya Singh	20	1	Manager	90000	2020-01-10
4	Rahul Mehta	20	3	Analyst	55000	2021-09-20
5	Sara Khan	10	2	Analyst	52000	2022-02-05

▸ departments (sample rows)

DEPARTMENT_ID	DEPARTMENT_NAME	LOCATION
10	Engineering	New York
20	Sales	Chicago
30	HR	Boston

▸ customers & orders (sample rows)

CUSTOMER_ID	NAME	ORDER_ID	ORDER_DATE	AMOUNT
101	Alice Brown	5001	2024-01-12	250.00
101	Alice Brown	5004	2024-03-02	180.50
102	Mike Lee	5002	2024-01-20	89.99
103	Wei Zhang	5003	2024-02-15	540.00

▸ products (sample rows)

PRODUCT_ID	PRODUCT_NAME	CATEGORY_ID	PRICE	STOCK_QUANTITY
201	Wireless Mouse	1	25.99	120
202	Mechanical Keyboard	1	79.99	0
203	Office Chair	2	199.00	40

Contents

Sixteen topic sections take you from foundational filtering all the way to capstone, senior-level problems that combine recursion, windowing, and multi-table joins in a single query.

01	Basic Filtering & Aggregation	Q1–Q25
02	Joins-Based Problems	Q26–Q54
03	Duplicates, Ranking & Nth Value	Q55–Q83
04	Window Functions in Practice	Q84–Q112
05	Date & Time Problems	Q113–Q132
06	GROUP BY / HAVING Puzzles	Q133–Q152
07	Subqueries & CTEs	Q153–Q172
08	String & Pattern Problems	Q173–Q187
09	Real Interview Favorites / Hard	Q188–Q200
10	Advanced Window Functions & Analytics	Q201–Q225
11	Advanced Recursive CTEs & Hierarchies	Q226–Q245
12	Complex JOIN Patterns	Q246–Q270
13	Advanced Aggregation Pipelines	Q271–Q290
14	Schema Modifications & DDL	Q291–Q315
15	Performance Tuning & Query Optimization	Q316–Q335
16	Senior-Level Interview Challenges	Q336–Q350

01 Basic Filtering & Aggregation

25 questions · Q1–Q25

Foundational SELECT, WHERE, GROUP BY and aggregate-function problems every SQL practitioner must master.

Q1 Find employees earning more than the company average salary.

```
SELECT *
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Q2 Count the number of employees in each department.

```
SELECT department_id, COUNT(*) AS total_employees
FROM employees
GROUP BY department_id;
```

Q3 Find employees who do not have a manager assigned.

```
SELECT *
FROM employees
WHERE manager_id IS NULL;
```

Q4 Find all employees whose email is missing (NULL).

```
SELECT *
FROM employees
WHERE email IS NULL;
```

01 Basic Filtering & Aggregation

CONTINUED · Q5–Q8

Q5 Find employees hired in the current year.

```
SELECT *
FROM employees
WHERE EXTRACT(YEAR FROM join_date) = EXTRACT(YEAR FROM CURRENT_DATE);
```

Q6 Find the top 3 highest paid employees in each department.

```
SELECT *
FROM (
  SELECT e.*,
         ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary
DESC) AS rnk
  FROM employees e
) ranked
WHERE rnk <= 3;
```

Q7 Find employees whose name starts with 'A'.

```
SELECT *
FROM employees
WHERE name LIKE 'A%';
```

Q8 Find employees whose name ends with 'n'.

```
SELECT *
FROM employees
WHERE name LIKE '%n';
```

01 Basic Filtering & Aggregation

CONTINUED · Q9–Q12

Q9 Find the total salary expense per department.

```
SELECT department_id, SUM(salary) AS total_salary
FROM employees
GROUP BY department_id;
```

Q10 Find the highest and lowest salary in the company.

```
SELECT MAX(salary) AS highest_salary, MIN(salary) AS lowest_salary
FROM employees;
```

Q11 Find employees who earn between 40000 and 80000.

```
SELECT *
FROM employees
WHERE salary BETWEEN 40000 AND 80000;
```

Q12 Find the number of distinct job titles in the company.

```
SELECT COUNT(DISTINCT job_title) AS distinct_titles
FROM employees;
```

01 Basic Filtering & Aggregation

CONTINUED · Q13–Q16

Q13 Find employees whose salary is not NULL and greater than zero.

```
SELECT *
FROM employees
WHERE salary IS NOT NULL AND salary > 0;
```

Q14 Find all employees sorted by salary in descending order.

```
SELECT *
FROM employees
ORDER BY salary DESC;
```

Q15 Find the second lowest salary in the employees table.

```
SELECT MIN(salary) AS second_lowest
FROM employees
WHERE salary > (SELECT MIN(salary) FROM employees);
```

Q16 Find departments having an average salary above 50000.

```
SELECT department_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY department_id
HAVING AVG(salary) > 50000;
```

01 Basic Filtering & Aggregation

CONTINUED · Q17–Q20

Q17 Find employees whose job title is either 'Manager' or 'Director'.

```
SELECT *
FROM employees
WHERE job_title IN ('Manager','Director');
```

Q18 Find employees who joined between two given dates.

```
SELECT *
FROM employees
WHERE join_date BETWEEN '2023-01-01' AND '2023-12-31';
```

Q19 Count how many employees have a salary above 100000.

```
SELECT COUNT(*) AS high_earners
FROM employees
WHERE salary > 100000;
```

Q20 Find the average salary rounded to two decimal places.

```
SELECT ROUND(AVG(salary), 2) AS avg_salary
FROM employees;
```

01 Basic Filtering & Aggregation

CONTINUED · Q21–Q24

Q21 List unique department IDs from the employees table.

```
SELECT DISTINCT department_id
FROM employees;
```

Q22 Find employees with a salary exactly equal to the maximum salary.

```
SELECT *
FROM employees
WHERE salary = (SELECT MAX(salary) FROM employees);
```

Q23 Find the total number of employees in the company.

```
SELECT COUNT(*) AS total_employees
FROM employees;
```

Q24 Find employees whose name contains the substring 'an'.

```
SELECT *
FROM employees
WHERE name LIKE '%an%';
```

01 Basic Filtering & Aggregation

CONTINUED · Q25–Q25

Q25 Find the 5 most recently hired employees.

```
SELECT *
FROM employees
ORDER BY join_date DESC
LIMIT 5;
```

02 Joins-Based Problems

29 questions · Q26–Q54

Multi-table problems testing INNER, LEFT, RIGHT and SELF joins on real-world schemas.

Q26 Find employees who are not assigned to any department.

```
SELECT e.*
FROM employees e
LEFT JOIN departments d ON e.department_id = d.department_id
WHERE d.department_id IS NULL;
```

Q27 Find customers who have never placed an order.

```
SELECT c.*
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Q28 Find products that have never been sold.

```
SELECT p.*
FROM products p
LEFT JOIN order_items oi ON p.product_id = oi.product_id
WHERE oi.product_id IS NULL;
```

Q29 Display each employee along with their manager's name.

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.id;
```

02 Joins-Based Problems

CONTINUED · Q30–Q33

Q30 Find departments along with their employee count, including departments with zero employees.

```
SELECT d.department_name, COUNT(e.id) AS employee_count
FROM departments d
LEFT JOIN employees e ON d.department_id = e.department_id
GROUP BY d.department_name;
```

Q31 Find students enrolled in more than 2 courses.

```
SELECT student_id, COUNT(course_id) AS total_courses
FROM enrollments
GROUP BY student_id
HAVING COUNT(course_id) > 2;
```

Q32 Find orphaned order_items rows whose product no longer exists.

```
SELECT oi.*
FROM order_items oi
LEFT JOIN products p ON oi.product_id = p.product_id
WHERE p.product_id IS NULL;
```

Q33 Find pairs of employees who work in the same department.

```
SELECT e1.name AS employee1, e2.name AS employee2, e1.department_id
FROM employees e1
JOIN employees e2
  ON e1.department_id = e2.department_id
 AND e1.id < e2.id;
```

02 Joins-Based Problems

CONTINUED · Q34–Q37

Q34 Find customers who placed orders in 2024 along with their total spend.

```
SELECT c.customer_id, c.name, SUM(o.amount) AS total_spend
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE EXTRACT(YEAR FROM o.order_date) = 2024
GROUP BY c.customer_id, c.name;
```

Q35 Find products along with the category name they belong to.

```
SELECT p.product_name, c.category_name
FROM products p
JOIN categories c ON p.category_id = c.category_id;
```

Q36 Find employees who work in departments located in 'New York'.

```
SELECT e.*
FROM employees e
JOIN departments d ON e.department_id = d.department_id
WHERE d.location = 'New York';
```

Q37 Find the manager with the most direct reports.

```
SELECT manager_id, COUNT(*) AS direct_reports
FROM employees
WHERE manager_id IS NOT NULL
GROUP BY manager_id
ORDER BY direct_reports DESC
LIMIT 1;
```

02 Joins-Based Problems

CONTINUED · Q38–Q41

Q38 Find all orders along with customer name and product name.

```
SELECT o.order_id, c.name AS customer, p.product_name
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id;
```

Q39 Find employees who share the same job title as 'John'.

```
SELECT e2.*
FROM employees e1
JOIN employees e2 ON e1.job_title = e2.job_title
WHERE e1.name = 'John' AND e2.name <> 'John';
```

Q40 Find departments that have at least one employee earning above 90000.

```
SELECT DISTINCT d.department_name
FROM departments d
JOIN employees e ON d.department_id = e.department_id
WHERE e.salary > 90000;
```

Q41 Find customers who ordered every product in a given category.

```
SELECT o.customer_id
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
WHERE p.category_id = 1
GROUP BY o.customer_id
HAVING COUNT(DISTINCT p.product_id) = (
  SELECT COUNT(*) FROM products WHERE category_id = 1
);
```

02 Joins-Based Problems

CONTINUED · Q42–Q45

Q42 Find employees and the total number of projects each is assigned to.

```
SELECT e.name, COUNT(pa.project_id) AS total_projects
FROM employees e
LEFT JOIN project_assignments pa ON e.id = pa.employee_id
GROUP BY e.name;
```

Q43 Find vendors who supply products that are currently out of stock.

```
SELECT DISTINCT v.vendor_name
FROM vendors v
JOIN products p ON v.vendor_id = p.vendor_id
WHERE p.stock_quantity = 0;
```

Q44 Find the department with the highest total salary expense.

```
SELECT d.department_name, SUM(e.salary) AS total_salary
FROM departments d
JOIN employees e ON d.department_id = e.department_id
GROUP BY d.department_name
ORDER BY total_salary DESC
LIMIT 1;
```

Q45 Find customers who have placed orders for products costing more than 500.

```
SELECT DISTINCT c.customer_id, c.name
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
WHERE p.price > 500;
```

02 Joins-Based Problems

CONTINUED · Q46–Q49

Q46 Find employees who joined before their manager.

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
JOIN employees m ON e.manager_id = m.id
WHERE e.join_date < m.join_date;
```

Q47 Find all combinations of employees and departments using a cross join (for testing).

```
SELECT e.name, d.department_name
FROM employees e
CROSS JOIN departments d;
```

Q48 Find suppliers who supply more than one category of product.

```
SELECT vendor_id
FROM products
GROUP BY vendor_id
HAVING COUNT(DISTINCT category_id) > 1;
```

Q49 Find the employee count for each manager, including managers with zero reports.

```
SELECT m.id, m.name, COUNT(e.id) AS reports
FROM employees m
LEFT JOIN employees e ON e.manager_id = m.id
GROUP BY m.id, m.name;
```

02 Joins-Based Problems

CONTINUED · Q50–Q53

Q50 Find customers whose total order amount exceeds 10000.

```
SELECT c.customer_id, c.name, SUM(o.amount) AS total_spend
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name
HAVING SUM(o.amount) > 10000;
```

Q51 Find products purchased by the same customers who bought product X.

```
SELECT DISTINCT oi2.product_id
FROM order_items oi1
JOIN orders o1 ON oi1.order_id = o1.order_id
JOIN orders o2 ON o1.customer_id = o2.customer_id
JOIN order_items oi2 ON o2.order_id = oi2.order_id
WHERE oi1.product_id = 'X' AND oi2.product_id <> 'X';
```

Q52 Find employees who have never been assigned to any project.

```
SELECT e.*
FROM employees e
LEFT JOIN project_assignments pa ON e.id = pa.employee_id
WHERE pa.employee_id IS NULL;
```

Q53 Find the average order amount per customer along with customer name.

```
SELECT c.name, AVG(o.amount) AS avg_order_amount
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.name;
```

02 Joins-Based Problems

CONTINUED · Q54–Q54

Q54 Find departments with more managers than regular employees.

```
SELECT department_id
FROM employees
GROUP BY department_id
HAVING SUM(CASE WHEN job_title='Manager' THEN 1 ELSE 0 END) >
       SUM(CASE WHEN job_title<>'Manager' THEN 1 ELSE 0 END);
```

03 Duplicates, Ranking & Nth Value

29 questions · Q55–Q83

The most frequently asked interview pattern: finding duplicates and Nth highest/lowest values using several techniques.

Q55 Find the second highest salary from the employees table.

```
SELECT MAX(salary) AS SecondHighestSalary
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Q56 Find the third highest salary using a window function.

```
SELECT salary
FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM employees
) ranked
WHERE rnk = 3
LIMIT 1;
```

Q57 Find the Nth highest salary using a generic, parameterised query.

```
SELECT DISTINCT salary
FROM employees e1
WHERE (N - 1) = (
  SELECT COUNT(DISTINCT salary)
  FROM employees e2
  WHERE e2.salary > e1.salary
);
```

Q58 Find duplicate records based on the employee name.

```
SELECT name, COUNT(*) AS occurrences
FROM employees
GROUP BY name
HAVING COUNT(*) > 1;
```

03 Duplicates, Ranking & Nth Value

CONTINUED · Q59–Q62

Q59 Delete duplicate rows keeping only one copy.

```
DELETE FROM employees
WHERE id NOT IN (
  SELECT MIN(id)
  FROM employees
  GROUP BY name, email
);
```

Q60 Find duplicate emails along with the number of times they occur.

```
SELECT email, COUNT(*) AS total
FROM employees
GROUP BY email
HAVING COUNT(*) > 1;
```

Q61 Rank employees by salary within each department.

```
SELECT name, department_id, salary,
       RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) AS
dept_rank
FROM employees;
```

Q62 Find ties in salary ranking using DENSE_RANK.

```
SELECT name, salary,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
FROM employees;
```

03 Duplicates, Ranking & Nth Value

CONTINUED · Q63–Q66

Q63 Find the lowest salary in each department.

```
SELECT department_id, MIN(salary) AS lowest_salary
FROM employees
GROUP BY department_id;
```

Q64 Find the second highest salary per department.

```
SELECT department_id, salary
FROM (
  SELECT department_id, salary,
         DENSE_RANK() OVER (PARTITION BY department_id ORDER BY salary
DESC) AS rnk
  FROM employees
) ranked
WHERE rnk = 2;
```

Q65 Find rows that are exact duplicates across all columns.

```
SELECT name, email, salary, department_id, COUNT(*)
FROM employees
GROUP BY name, email, salary, department_id
HAVING COUNT(*) > 1;
```

Q66 Find employees with a unique (non-duplicated) salary value.

```
SELECT salary
FROM employees
GROUP BY salary
HAVING COUNT(*) = 1;
```

03 Duplicates, Ranking & Nth Value

CONTINUED · Q67–Q70

Q67 Assign a row number to each employee ordered by hire date.

```
SELECT name, join_date,
       ROW_NUMBER() OVER (ORDER BY join_date) AS hire_sequence
FROM employees;
```

Q68 Find the top 1 highest paid employee per job title using RANK.

```
SELECT *
FROM (
  SELECT *, RANK() OVER (PARTITION BY job_title ORDER BY salary DESC) AS
rnk
  FROM employees
) ranked
WHERE rnk = 1;
```

Q69 Remove duplicate customer records keeping the most recently updated one.

```
DELETE FROM customers
WHERE customer_id NOT IN (
  SELECT customer_id FROM (
    SELECT customer_id,
           ROW_NUMBER() OVER (PARTITION BY email ORDER BY updated_at DESC)
AS rn
    FROM customers
  ) t
  WHERE rn = 1
);
```

Q70 Find the Nth highest order amount per customer.

```
SELECT customer_id, amount
FROM (
  SELECT customer_id, amount,
         DENSE_RANK() OVER (PARTITION BY customer_id ORDER BY amount DESC)
AS rnk
  FROM orders
) ranked
WHERE rnk = 2;
```

03 Duplicates, Ranking & Nth Value

CONTINUED · Q71–Q74

Q71 Find duplicate phone numbers shared by multiple customers.

```
SELECT phone, COUNT(*) AS total
FROM customers
GROUP BY phone
HAVING COUNT(*) > 1;
```

Q72 Use NTILE to split employees into 4 salary quartiles.

```
SELECT name, salary,
       NTILE(4) OVER (ORDER BY salary) AS quartile
FROM employees;
```

Q73 Find the second most expensive product in each category.

```
SELECT category_id, product_name, price
FROM (
  SELECT category_id, product_name, price,
         DENSE_RANK() OVER (PARTITION BY category_id ORDER BY price DESC)
AS rnk
  FROM products
) ranked
WHERE rnk = 2;
```

Q74 Find employees whose salary matches another employee's salary exactly.

```
SELECT e1.name, e1.salary
FROM employees e1
JOIN employees e2
  ON e1.salary = e2.salary AND e1.id <> e2.id;
```

03 Duplicates, Ranking & Nth Value

CONTINUED · Q75–Q78

Q75 Identify and keep only the latest order per customer, deleting older duplicates of the same product.

```
DELETE FROM orders
WHERE order_id NOT IN (
  SELECT order_id FROM (
    SELECT order_id,
           ROW_NUMBER() OVER (PARTITION BY customer_id, product_id ORDER
BY order_date DESC) AS rn
    FROM orders
  ) t
  WHERE rn = 1
);
```

Q76 Find the rank of a specific employee's salary within the company.

```
SELECT name, salary,
       RANK() OVER (ORDER BY salary DESC) AS overall_rank
FROM employees
WHERE name = 'John';
```

Q77 Find duplicate rows in a transactions table based on amount and date.

```
SELECT amount, transaction_date, COUNT(*)
FROM transactions
GROUP BY amount, transaction_date
HAVING COUNT(*) > 1;
```

Q78 Find the highest salary that is not unique (shared by more than one employee).

```
SELECT salary
FROM employees
GROUP BY salary
HAVING COUNT(*) > 1
ORDER BY salary DESC
LIMIT 1;
```

03 Duplicates, Ranking & Nth Value

CONTINUED · Q79–Q82

Q79 Find the bottom 3 performing products by total sales using ranking.

```
SELECT product_id, total_sales
FROM (
  SELECT product_id, SUM(quantity * price) AS total_sales,
         RANK() OVER (ORDER BY SUM(quantity * price) ASC) AS rnk
  FROM order_items
  GROUP BY product_id
) ranked
WHERE rnk <= 3;
```

Q80 Find duplicate combinations of student and course in an enrollments table.

```
SELECT student_id, course_id, COUNT(*)
FROM enrollments
GROUP BY student_id, course_id
HAVING COUNT(*) > 1;
```

Q81 Find the percentile rank of each employee's salary.

```
SELECT name, salary,
       PERCENT_RANK() OVER (ORDER BY salary) AS percentile_rank
FROM employees;
```

Q82 Find employees who have the same salary as the department's average.

```
SELECT e.*
FROM employees e
JOIN (
  SELECT department_id, AVG(salary) AS avg_salary
  FROM employees
  GROUP BY department_id
) d ON e.department_id = d.department_id AND e.salary = d.avg_salary;
```

03 Duplicates, Ranking & Nth Value

CONTINUED · Q83–Q83

Q83 Keep only the first occurrence of each duplicate invoice number.

```
DELETE FROM invoices
WHERE invoice_id NOT IN (
  SELECT MIN(invoice_id)
  FROM invoices
  GROUP BY invoice_number
);
```

04 Window Functions in Practice

29 questions · Q84–Q112

Running totals, moving averages, gaps-and-islands, and other analytics-style problems solved with OVER().

Q84 Calculate a running total of sales ordered by date.

```
SELECT sale_date, amount,
       SUM(amount) OVER (ORDER BY sale_date) AS running_total
FROM sales;
```

Q85 Calculate a 3-order moving average of order amounts per customer.

```
SELECT customer_id, order_date, amount,
       AVG(amount) OVER (
         PARTITION BY customer_id
         ORDER BY order_date
         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
       ) AS moving_avg
FROM orders;
```

Q86 Find the difference between each order amount and the previous order amount for the same customer.

```
SELECT customer_id, order_date, amount,
       amount - LAG(amount) OVER (PARTITION BY customer_id ORDER BY
order_date) AS diff
FROM orders;
```

Q87 Find the first and last order date for each customer.

```
SELECT customer_id,
       MIN(order_date) AS first_order,
       MAX(order_date) AS last_order
FROM orders
GROUP BY customer_id;
```

04 Window Functions in Practice

CONTINUED · Q88–Q91

Q88 Find the cumulative distribution of employee salaries.

```
SELECT name, salary,
       CUME_DIST() OVER (ORDER BY salary) AS cum_dist
FROM employees;
```

Q89 Calculate year-over-year revenue growth.

```
SELECT year, revenue,
       revenue - LAG(revenue) OVER (ORDER BY year) AS yoy_growth
FROM annual_revenue;
```

Q90 Find the running count of orders placed by each customer.

```
SELECT customer_id, order_date,
       COUNT(*) OVER (PARTITION BY customer_id ORDER BY order_date) AS
order_sequence
FROM orders;
```

Q91 Find the next order amount for each customer using LEAD.

```
SELECT customer_id, order_date, amount,
       LEAD(amount) OVER (PARTITION BY customer_id ORDER BY order_date) AS
next_amount
FROM orders;
```

04 Window Functions in Practice

CONTINUED · Q92–Q95

Q92 Find the highest salary so far for each employee ordered by date of raise.

```
SELECT employee_id, raise_date, salary,
       MAX(salary) OVER (PARTITION BY employee_id ORDER BY raise_date) AS
highest_so_far
FROM salary_history;
```

Q93 Calculate the percentage contribution of each product's sales to the total sales.

```
SELECT product_id, total_sales,
       ROUND(100.0 * total_sales / SUM(total_sales) OVER (), 2) AS
pct_of_total
FROM (
  SELECT product_id, SUM(quantity * price) AS total_sales
  FROM order_items
  GROUP BY product_id
) t;
```

Q94 Find the 3rd highest salary per department using NTH_VALUE.

```
SELECT department_id,
       NTH_VALUE(salary, 3) OVER (
         PARTITION BY department_id ORDER BY salary DESC
         ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
       ) AS third_highest
FROM employees;
```

Q95 Find the running maximum stock price per day.

```
SELECT trade_date, price,
       MAX(price) OVER (ORDER BY trade_date) AS running_max
FROM stock_prices;
```

04 Window Functions in Practice

CONTINUED · Q96–Q99

Q96 Identify the rank of each customer based on total spend, without gaps.

```
SELECT customer_id, total_spend,
       DENSE_RANK() OVER (ORDER BY total_spend DESC) AS spend_rank
FROM (
  SELECT customer_id, SUM(amount) AS total_spend
  FROM orders
  GROUP BY customer_id
) t;
```

Q97 Find each employee's salary compared to the department average using window functions.

```
SELECT name, department_id, salary,
       salary - AVG(salary) OVER (PARTITION BY department_id) AS
diff_from_avg
FROM employees;
```

Q98 Calculate the running total of expenses reset at the start of every month.

```
SELECT expense_date, amount,
       SUM(amount) OVER (
         PARTITION BY DATE_TRUNC('month', expense_date)
         ORDER BY expense_date
       ) AS monthly_running_total
FROM expenses;
```

Q99 Find the previous and next salary of each employee ordered by salary.

```
SELECT name, salary,
       LAG(salary) OVER (ORDER BY salary) AS prev_salary,
       LEAD(salary) OVER (ORDER BY salary) AS next_salary
FROM employees;
```

04 Window Functions in Practice

CONTINUED · Q100–Q103

Q100 Find the percentage change in monthly sales.

```
SELECT month, total_sales,
       ROUND(100.0 * (total_sales - LAG(total_sales) OVER (ORDER BY
month))
             / LAG(total_sales) OVER (ORDER BY month), 2) AS pct_change
FROM monthly_sales;
```

Q101 Identify the first purchase made by each customer using window functions.

```
SELECT *
FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY
order_date) AS rn
  FROM orders
) t
WHERE rn = 1;
```

Q102 Find a 7-day rolling average of daily website visits.

```
SELECT visit_date, visits,
       AVG(visits) OVER (
         ORDER BY visit_date
         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
       ) AS rolling_7day_avg
FROM daily_visits;
```

Q103 Find the gap in days between consecutive orders for each customer.

```
SELECT customer_id, order_date,
       order_date - LAG(order_date) OVER (PARTITION BY customer_id ORDER
BY order_date) AS days_gap
FROM orders;
```

04 Window Functions in Practice

CONTINUED · Q104–Q107

Q104 Rank products within each category by total revenue and keep only the top 2.

```
SELECT *
FROM (
  SELECT category_id, product_id, total_revenue,
         RANK() OVER (PARTITION BY category_id ORDER BY total_revenue
DESC) AS rnk
  FROM (
    SELECT p.category_id, oi.product_id, SUM(oi.quantity * oi.price) AS
total_revenue
    FROM order_items oi
    JOIN products p ON oi.product_id = p.product_id
    GROUP BY p.category_id, oi.product_id
  ) rev
) ranked
WHERE rnk <= 2;
```

Q105 Find the salary growth of each employee compared to their first recorded salary.

```
SELECT employee_id, raise_date, salary,
       salary - FIRST_VALUE(salary) OVER (
         PARTITION BY employee_id ORDER BY raise_date
       ) AS growth_since_start
FROM salary_history;
```

Q106 Find the median order amount using PERCENTILE_CONT.

```
SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY amount) AS
median_amount
FROM orders;
```

Q107 Find customers whose spending increased in every consecutive quarter.

```
SELECT customer_id
FROM (
  SELECT customer_id, quarter, amount,
         LAG(amount) OVER (PARTITION BY customer_id ORDER BY quarter) AS
prev_amount
  FROM quarterly_spend
) t
GROUP BY customer_id
HAVING SUM(CASE WHEN amount <= prev_amount THEN 1 ELSE 0 END) = 0;
```

04 Window Functions in Practice

CONTINUED · Q108–Q111

Q108 Find the average time gap between orders for each customer.

```
SELECT customer_id, AVG(gap_days) AS avg_gap
FROM (
  SELECT customer_id,
         order_date - LAG(order_date) OVER (PARTITION BY customer_id ORDER
BY order_date) AS gap_days
  FROM orders
) t
GROUP BY customer_id;
```

Q109 Find the top performing salesperson for each month.

```
SELECT *
FROM (
  SELECT month, salesperson_id, total_sales,
         RANK() OVER (PARTITION BY month ORDER BY total_sales DESC) AS rnk
  FROM monthly_sales_by_rep
) ranked
WHERE rnk = 1;
```

Q110 Find the cumulative number of new customers acquired each month.

```
SELECT signup_month, new_customers,
       SUM(new_customers) OVER (ORDER BY signup_month) AS
cumulative_customers
FROM (
  SELECT DATE_TRUNC('month', signup_date) AS signup_month, COUNT(*) AS
new_customers
  FROM customers
  GROUP BY DATE_TRUNC('month', signup_date)
) t;
```

Q111 Calculate the variance of each employee's salary from the company average using window functions.

```
SELECT name, salary,
       salary - AVG(salary) OVER () AS variance_from_company_avg
FROM employees;
```

04 Window Functions in Practice

CONTINUED · Q112–Q112

Q112 Find the last value of stock price in each trading day partitioned by stock symbol.

```
SELECT symbol, trade_date, price,
       LAST_VALUE(price) OVER (
         PARTITION BY symbol, trade_date ORDER BY trade_time
         ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
       ) AS closing_price
FROM stock_prices;
```

05 Date & Time Problems

20 questions · Q113–Q132

Handling temporal data: streaks, gaps, age calculations and time-bucketed aggregations.

Q113 Find employees who joined in the last 6 months.

```
SELECT *
FROM employees
WHERE join_date >= CURRENT_DATE - INTERVAL '6 months';
```

Q114 Find customers who logged in on consecutive days (a login streak).

```
SELECT customer_id, login_date,
       login_date - ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY
login_date)::INT AS streak_group
FROM logins;
```

Q115 Find missing dates in a sequence of daily sales records.

```
SELECT d.calendar_date
FROM calendar d
LEFT JOIN sales s ON d.calendar_date = s.sale_date
WHERE s.sale_date IS NULL
  AND d.calendar_date BETWEEN (SELECT MIN(sale_date) FROM sales)
                          AND (SELECT MAX(sale_date) FROM sales);
```

Q116 Calculate each employee's age from their date of birth.

```
SELECT name, dob,
       EXTRACT(YEAR FROM AGE(CURRENT_DATE, dob)) AS age
FROM employees;
```

05 Date & Time Problems

CONTINUED · Q117–Q120

Q117 Find orders placed on weekends.

```
SELECT *
FROM orders
WHERE EXTRACT(DOW FROM order_date) IN (0, 6);
```

Q118 Find the number of monthly active users.

```
SELECT DATE_TRUNC('month', activity_date) AS month,
       COUNT(DISTINCT user_id) AS active_users
FROM user_activity
GROUP BY DATE_TRUNC('month', activity_date);
```

Q119 Find customers who have not placed an order in the last 90 days.

```
SELECT c.*
FROM customers c
WHERE c.customer_id NOT IN (
  SELECT customer_id FROM orders
  WHERE order_date >= CURRENT_DATE - INTERVAL '90 days'
);
```

Q120 Find the number of days between an employee's hire date and today.

```
SELECT name, join_date,
       CURRENT_DATE - join_date AS days_employed
FROM employees;
```

05 Date & Time Problems

CONTINUED · Q121–Q124

Q121 Find the busiest day of the week for orders.

```
SELECT TO_CHAR(order_date,'Day') AS day_of_week,
       COUNT(*) AS total_orders
FROM orders
GROUP BY TO_CHAR(order_date,'Day')
ORDER BY total_orders DESC
LIMIT 1;
```

Q122 Find employees whose work anniversary is this month.

```
SELECT *
FROM employees
WHERE EXTRACT(MONTH FROM join_date) = EXTRACT(MONTH FROM CURRENT_DATE);
```

Q123 Find the total sales for each quarter of the year.

```
SELECT EXTRACT(QUARTER FROM sale_date) AS quarter,
       SUM(amount) AS total_sales
FROM sales
GROUP BY EXTRACT(QUARTER FROM sale_date);
```

Q124 Find the longest streak of consecutive days a user was active.

```
SELECT user_id, MAX(streak_length) AS longest_streak
FROM (
  SELECT user_id, COUNT(*) AS streak_length
  FROM (
    SELECT user_id, activity_date,
           activity_date - ROW_NUMBER() OVER (PARTITION BY user_id ORDER
BY activity_date)::INT AS grp
    FROM user_activity
  ) t
  GROUP BY user_id, grp
) streaks
GROUP BY user_id;
```

05 Date & Time Problems

CONTINUED · Q125–Q128

Q125 Find the average number of days between consecutive purchases for each customer.

```
SELECT customer_id, AVG(days_between) AS avg_days_between
FROM (
  SELECT customer_id,
         order_date - LAG(order_date) OVER (PARTITION BY customer_id ORDER
BY order_date) AS days_between
  FROM orders
) t
GROUP BY customer_id;
```

Q126 Find all orders placed in the last 24 hours.

```
SELECT *
FROM orders
WHERE order_timestamp >= NOW() - INTERVAL '24 hours';
```

Q127 Find the month with the highest revenue.

```
SELECT DATE_TRUNC('month', sale_date) AS month,
       SUM(amount) AS total_revenue
FROM sales
GROUP BY DATE_TRUNC('month', sale_date)
ORDER BY total_revenue DESC
LIMIT 1;
```

Q128 Find employees who have completed exactly 5 years of service.

```
SELECT *
FROM employees
WHERE join_date = CURRENT_DATE - INTERVAL '5 years';
```

05 Date & Time Problems

CONTINUED · Q129–Q132

Q129 Convert a UTC timestamp column to a specific time zone.

```
SELECT order_id, order_timestamp AT TIME ZONE 'UTC' AT TIME ZONE
'Asia/Kolkata' AS local_time
FROM orders;
```

Q130 Find users who signed up but never logged in within 7 days.

```
SELECT u.*
FROM users u
LEFT JOIN logins l ON u.user_id = l.user_id
  AND l.login_date BETWEEN u.signup_date AND u.signup_date + INTERVAL '7
days'
WHERE l.user_id IS NULL;
```

Q131 Find the number of new customers acquired each week.

```
SELECT DATE_TRUNC('week', signup_date) AS week, COUNT(*) AS new_customers
FROM customers
GROUP BY DATE_TRUNC('week', signup_date);
```

Q132 Find orders that took more than 5 days to be shipped after being placed.

```
SELECT *
FROM orders
WHERE shipped_date - order_date > 5;
```

06 GROUP BY / HAVING Puzzles

20 questions · Q133–Q152

Aggregation edge cases that frequently trip up candidates in real interviews.

Q133 Find departments having more than 5 employees.

```
SELECT department_id, COUNT(*) AS num_employees
FROM employees
GROUP BY department_id
HAVING COUNT(*) > 5;
```

Q134 Find products ordered by more than 10 distinct customers.

```
SELECT oi.product_id, COUNT(DISTINCT o.customer_id) AS distinct_customers
FROM order_items oi
JOIN orders o ON oi.order_id = o.order_id
GROUP BY oi.product_id
HAVING COUNT(DISTINCT o.customer_id) > 10;
```

Q135 Find departments whose average salary is above the company average.

```
SELECT department_id, AVG(salary) AS dept_avg
FROM employees
GROUP BY department_id
HAVING AVG(salary) > (SELECT AVG(salary) FROM employees);
```

Q136 Find categories that have zero sales.

```
SELECT c.category_id, c.category_name
FROM categories c
LEFT JOIN products p ON c.category_id = p.category_id
LEFT JOIN order_items oi ON p.product_id = oi.product_id
GROUP BY c.category_id, c.category_name
HAVING SUM(oi.quantity) IS NULL;
```

06 GROUP BY / HAVING Puzzles

CONTINUED · Q137–Q140

Q137 Find customers who have placed orders worth more than 5000 in total.

```
SELECT customer_id, SUM(amount) AS total_spent
FROM orders
GROUP BY customer_id
HAVING SUM(amount) > 5000;
```

Q138 Find job titles held by more than one employee.

```
SELECT job_title, COUNT(*) AS total
FROM employees
GROUP BY job_title
HAVING COUNT(*) > 1;
```

Q139 Find departments where the maximum salary is less than 60000.

```
SELECT department_id, MAX(salary) AS max_salary
FROM employees
GROUP BY department_id
HAVING MAX(salary) < 60000;
```

Q140 Find students who have failed more than 2 subjects.

```
SELECT student_id, COUNT(*) AS failed_subjects
FROM results
WHERE grade = 'F'
GROUP BY student_id
HAVING COUNT(*) > 2;
```

06 GROUP BY / HAVING Puzzles

CONTINUED · Q141–Q144

Q141 Find customers who ordered the same product more than 3 times.

```
SELECT customer_id, product_id, COUNT(*) AS times_ordered
FROM order_items oi
JOIN orders o ON oi.order_id = o.order_id
GROUP BY customer_id, product_id
HAVING COUNT(*) > 3;
```

Q142 Find departments with a total headcount between 10 and 20.

```
SELECT department_id, COUNT(*) AS headcount
FROM employees
GROUP BY department_id
HAVING COUNT(*) BETWEEN 10 AND 20;
```

Q143 Find vendors supplying products with an average price above 1000.

```
SELECT vendor_id, AVG(price) AS avg_price
FROM products
GROUP BY vendor_id
HAVING AVG(price) > 1000;
```

Q144 Find months where total sales dropped below 10000.

```
SELECT DATE_TRUNC('month', sale_date) AS month,
       SUM(amount) AS total_sales
FROM sales
GROUP BY DATE_TRUNC('month', sale_date)
HAVING SUM(amount) < 10000;
```

06 GROUP BY / HAVING Puzzles

CONTINUED · Q145–Q148

Q145 Find employees grouped by job title having a combined salary above 500000.

```
SELECT job_title, SUM(salary) AS total_salary
FROM employees
GROUP BY job_title
HAVING SUM(salary) > 500000;
```

Q146 Find product categories with fewer than 5 products listed.

```
SELECT category_id, COUNT(*) AS total_products
FROM products
GROUP BY category_id
HAVING COUNT(*) < 5;
```

Q147 Find customers with orders in more than 3 different months.

```
SELECT customer_id, COUNT(DISTINCT DATE_TRUNC('month', order_date)) AS
active_months
FROM orders
GROUP BY customer_id
HAVING COUNT(DISTINCT DATE_TRUNC('month', order_date)) > 3;
```

Q148 Find regions where total revenue exceeds 1 million.

```
SELECT region, SUM(amount) AS total_revenue
FROM sales
GROUP BY region
HAVING SUM(amount) > 1000000;
```

06 GROUP BY / HAVING Puzzles

CONTINUED · Q149–Q152

Q149 Find suppliers providing only a single product.

```
SELECT vendor_id, COUNT(*) AS total_products
FROM products
GROUP BY vendor_id
HAVING COUNT(*) = 1;
```

Q150 Find departments where the salary range (max-min) exceeds 50000.

```
SELECT department_id, MAX(salary) - MIN(salary) AS salary_range
FROM employees
GROUP BY department_id
HAVING MAX(salary) - MIN(salary) > 50000;
```

Q151 Find customers whose average order value is below 100.

```
SELECT customer_id, AVG(amount) AS avg_order_value
FROM orders
GROUP BY customer_id
HAVING AVG(amount) < 100;
```

Q152 Find employees grouped by department who collectively worked more than 1000 overtime hours.

```
SELECT department_id, SUM(overtime_hours) AS total_overtime
FROM employees
GROUP BY department_id
HAVING SUM(overtime_hours) > 1000;
```

07 Subqueries & CTEs

20 questions · Q153–Q172

Correlated subqueries, recursive CTEs for hierarchies, and statistical queries like median and mode.

Q153 Find employees who earn more than their manager.

```
SELECT e.name AS Employee, e.salary, m.name AS Manager, m.salary AS
ManagerSalary
FROM employees e
JOIN employees m ON e.manager_id = m.id
WHERE e.salary > m.salary;
```

Q154 Build the full organizational hierarchy using a recursive CTE.

```
WITH RECURSIVE org_chart AS (
  SELECT id, name, manager_id, 1 AS level
  FROM employees
  WHERE manager_id IS NULL
  UNION ALL
  SELECT e.id, e.name, e.manager_id, oc.level + 1
  FROM employees e
  JOIN org_chart oc ON e.manager_id = oc.id
)
SELECT * FROM org_chart
ORDER BY level;
```

Q155 Find the median salary in the employees table.

```
SELECT AVG(salary) AS median_salary
FROM (
  SELECT salary
  FROM employees
  ORDER BY salary
  LIMIT 2 - (SELECT COUNT(*) FROM employees) % 2
  OFFSET (SELECT (COUNT(*) - 1) / 2 FROM employees)
) AS median_subquery;
```

Q156 Find the mode (most frequently occurring salary).

```
SELECT salary
FROM employees
GROUP BY salary
ORDER BY COUNT(*) DESC
LIMIT 1;
```

07 Subqueries & CTEs

CONTINUED · Q157–Q160

Q157 Find each employee's level/depth in the reporting hierarchy.

```
WITH RECURSIVE hierarchy AS (
  SELECT id, name, manager_id, 0 AS depth
  FROM employees
  WHERE manager_id IS NULL
  UNION ALL
  SELECT e.id, e.name, e.manager_id, h.depth + 1
  FROM employees e
  JOIN hierarchy h ON e.manager_id = h.id
)
SELECT * FROM hierarchy;
```

Q158 Find customers whose total spend is greater than the average spend across all customers.

```
SELECT customer_id, total_spend
FROM (
  SELECT customer_id, SUM(amount) AS total_spend
  FROM orders
  GROUP BY customer_id
) c
WHERE total_spend > (
  SELECT AVG(total_spend) FROM (
    SELECT SUM(amount) AS total_spend FROM orders GROUP BY customer_id
  ) avg_sub
);
```

Q159 Use a CTE to simplify a multi-step sales summary report.

```
WITH monthly_totals AS (
  SELECT DATE_TRUNC('month', sale_date) AS month,
         SUM(amount) AS total
  FROM sales
  GROUP BY DATE_TRUNC('month', sale_date)
)
SELECT month, total,
       total - LAG(total) OVER (ORDER BY month) AS month_over_month_change
FROM monthly_totals;
```

Q160 Find products priced above the average price within their own category (correlated subquery).

```
SELECT p1.*
FROM products p1
WHERE p1.price > (
  SELECT AVG(p2.price)
  FROM products p2
  WHERE p2.category_id = p1.category_id
);
```

07 Subqueries & CTEs

CONTINUED · Q161–Q164

Q161 Find the total number of direct and indirect reports for each manager using a recursive CTE.

```
WITH RECURSIVE reports AS (
  SELECT id, manager_id FROM employees
  WHERE manager_id IS NOT NULL
  UNION ALL
  SELECT e.id, r.manager_id
  FROM employees e
  JOIN reports r ON e.manager_id = r.id
)
SELECT manager_id, COUNT(*) AS total_reports
FROM reports
GROUP BY manager_id;
```

Q162 Find employees who do not appear in the project_assignments table using NOT EXISTS.

```
SELECT *
FROM employees e
WHERE NOT EXISTS (
  SELECT 1 FROM project_assignments pa WHERE pa.employee_id = e.id
);
```

Q163 Find customers who bought a product also bought by customer 'Alice' (self-referencing subquery).

```
SELECT DISTINCT o2.customer_id
FROM orders o1
JOIN order_items oi1 ON o1.order_id = oi1.order_id
JOIN order_items oi2 ON oi1.product_id = oi2.product_id
JOIN orders o2 ON oi2.order_id = o2.order_id
WHERE o1.customer_id = (SELECT customer_id FROM customers WHERE name =
'Alice')
  AND o2.customer_id <> o1.customer_id;
```

Q164 Use a CTE to find the top spender in each region.

```
WITH regional_spend AS (
  SELECT region, customer_id, SUM(amount) AS total_spend,
         RANK() OVER (PARTITION BY region ORDER BY SUM(amount) DESC) AS
rnk
  FROM orders o
  JOIN customers c ON o.customer_id = c.customer_id
  GROUP BY region, customer_id
)
SELECT * FROM regional_spend WHERE rnk = 1;
```

07 Subqueries & CTEs

CONTINUED · Q165–Q168

Q165 Find the percentage of employees earning above the median salary.

```
WITH median_calc AS (
  SELECT AVG(salary) AS median_salary
  FROM (
    SELECT salary FROM employees ORDER BY salary
    LIMIT 2 - (SELECT COUNT(*) FROM employees) % 2
    OFFSET (SELECT (COUNT(*) - 1) / 2 FROM employees)
  ) sub
)
SELECT 100.0 * COUNT(*) / (SELECT COUNT(*) FROM employees) AS
pct_above_median
FROM employees, median_calc
WHERE employees.salary > median_calc.median_salary;
```

Q166 Find all subordinates (direct and indirect) of a given manager using a recursive CTE.

```
WITH RECURSIVE subordinates AS (
  SELECT id, name, manager_id
  FROM employees
  WHERE manager_id = 101
  UNION ALL
  SELECT e.id, e.name, e.manager_id
  FROM employees e
  JOIN subordinates s ON e.manager_id = s.id
)
SELECT * FROM subordinates;
```

Q167 Find departments where every employee earns above 50000 using NOT EXISTS.

```
SELECT DISTINCT department_id
FROM employees e1
WHERE NOT EXISTS (
  SELECT 1 FROM employees e2
  WHERE e2.department_id = e1.department_id AND e2.salary <= 50000
);
```

Q168 Find the cumulative count of orders using a CTE combined with a window function.

```
WITH daily_orders AS (
  SELECT order_date, COUNT(*) AS daily_count
  FROM orders
  GROUP BY order_date
)
SELECT order_date, daily_count,
       SUM(daily_count) OVER (ORDER BY order_date) AS cumulative_orders
FROM daily_orders;
```

07 Subqueries & CTEs

CONTINUED · Q169–Q172

Q169 Find products that have never appeared together in the same order (using a self-join subquery).

```
SELECT p1.product_id AS product_a, p2.product_id AS product_b
FROM products p1
CROSS JOIN products p2
WHERE p1.product_id < p2.product_id
  AND NOT EXISTS (
    SELECT 1 FROM order_items oi1
    JOIN order_items oi2 ON oi1.order_id = oi2.order_id
    WHERE oi1.product_id = p1.product_id AND oi2.product_id =
p2.product_id
  );
```

Q170 Find the average tenure (in years) of employees per department using a CTE.

```
WITH tenure AS (
  SELECT department_id,
         EXTRACT(YEAR FROM AGE(CURRENT_DATE, join_date)) AS years_employed
  FROM employees
)
SELECT department_id, AVG(years_employed) AS avg_tenure
FROM tenure
GROUP BY department_id;
```

Q171 Find employees earning the maximum salary within their reporting chain (their own subtree) using recursion.

```
WITH RECURSIVE chain AS (
  SELECT id, name, manager_id, salary, id AS root_manager
  FROM employees
  WHERE manager_id IS NULL
  UNION ALL
  SELECT e.id, e.name, e.manager_id, e.salary, c.root_manager
  FROM employees e
  JOIN chain c ON e.manager_id = c.id
)
SELECT root_manager, MAX(salary) AS max_salary_in_tree
FROM chain
GROUP BY root_manager;
```

Q172 Find customers whose most recent order amount is greater than their average order amount.

```
WITH customer_orders AS (
  SELECT customer_id, amount, order_date,
         AVG(amount) OVER (PARTITION BY customer_id) AS avg_amount,
         ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date
DESC) AS rn
  FROM orders
)
SELECT customer_id, amount, avg_amount
FROM customer_orders
WHERE rn = 1 AND amount > avg_amount;
```

08 String & Pattern Problems

15 questions · Q173–Q187

Text manipulation: extracting, transforming, and matching string-based data.

Q173 Find employees with duplicate names (case-insensitive).

```
SELECT LOWER(name) AS name_lower, COUNT(*)
FROM employees
GROUP BY LOWER(name)
HAVING COUNT(*) > 1;
```

Q174 Extract the domain name from each employee's email address.

```
SELECT email, SUBSTRING(email FROM POSITION('@' IN email) + 1) AS domain
FROM employees;
```

Q175 Find employees whose name contains only alphabetic characters.

```
SELECT *
FROM employees
WHERE name ~ '^[A-Za-z ]+$';
```

Q176 Concatenate first and last name into a single full name column.

```
SELECT first_name || ' ' || last_name AS full_name
FROM employees;
```

08 String & Pattern Problems

CONTINUED · Q177–Q180

Q177 Find the employee with the longest name.

```
SELECT name, LENGTH(name) AS name_length
FROM employees
ORDER BY LENGTH(name) DESC
LIMIT 1;
```

Q178 Find the employee with the shortest name.

```
SELECT name, LENGTH(name) AS name_length
FROM employees
ORDER BY LENGTH(name) ASC
LIMIT 1;
```

Q179 Convert all employee names to uppercase.

```
SELECT UPPER(name) AS name_upper
FROM employees;
```

Q180 Find employees whose email does not contain '@'.

```
SELECT *
FROM employees
WHERE email NOT LIKE '%@%';
```

08 String & Pattern Problems

CONTINUED · Q181–Q184

Q181 Find the count of vowels in each employee's name.

```
SELECT name,
       LENGTH(name) -
       LENGTH(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(
         LOWER(name),
         'a',''),'e',''),'i',''),'o',''),'u','')) AS vowel_count
FROM employees;
```

Q182 Find customers whose phone number does not match a 10-digit numeric pattern.

```
SELECT *
FROM customers
WHERE phone !~ '^[0-9]{10}$';
```

Q183 Trim leading and trailing spaces from product names.

```
SELECT TRIM(product_name) AS cleaned_name
FROM products;
```

Q184 Find the position of the substring 'mail' within each email address.

```
SELECT email, POSITION('mail' IN email) AS position_in_string
FROM employees;
```

08 String & Pattern Problems

CONTINUED · Q185–Q187

Q185 Replace all spaces in product names with underscores.

```
SELECT product_name, REPLACE(product_name,' ','_') AS slug
FROM products;
```

Q186 Find employees whose name has more than one word.

```
SELECT *
FROM employees
WHERE name LIKE '% %';
```

Q187 Reverse each customer's name.

```
SELECT name, REVERSE(name) AS reversed_name
FROM customers;
```

09 Real Interview Favorites / Hard

13 questions · Q188–Q200

Tricky, high-frequency interview puzzles: gaps-and-islands, pivoting, and complex existence checks.

Q188 Find the 3 consecutive numbers that are missing from an ID sequence.

```
SELECT id + 1 AS missing_start
FROM sequence_table s1
WHERE NOT EXISTS (SELECT 1 FROM sequence_table s2 WHERE s2.id = s1.id + 1)
  AND NOT EXISTS (SELECT 1 FROM sequence_table s3 WHERE s3.id = s1.id + 2)
  AND NOT EXISTS (SELECT 1 FROM sequence_table s4 WHERE s4.id = s1.id +
3);
```

Q189 Pivot monthly sales rows into columns (one column per month).

```
SELECT product_id,
       SUM(CASE WHEN month = 'Jan' THEN amount END) AS Jan,
       SUM(CASE WHEN month = 'Feb' THEN amount END) AS Feb,
       SUM(CASE WHEN month = 'Mar' THEN amount END) AS Mar
FROM monthly_sales
GROUP BY product_id;
```

Q190 Unpivot columns (Jan, Feb, Mar) back into rows.

```
SELECT product_id,'Jan' AS month, jan_sales AS amount
FROM sales_wide
UNION ALL
SELECT product_id,'Feb', feb_sales FROM sales_wide
UNION ALL
SELECT product_id,'Mar', mar_sales FROM sales_wide;
```

Q191 Find employees who never took a single day of leave.

```
SELECT e.*
FROM employees e
LEFT JOIN leave_records l ON e.id = l.employee_id
WHERE l.employee_id IS NULL;
```

09 Real Interview Favorites / Hard

CONTINUED · Q192–Q195

Q192 Find the top N products per category using window functions.

```
SELECT *
FROM (
  SELECT p.category_id, p.product_name, s.total_sales,
         ROW_NUMBER() OVER (PARTITION BY p.category_id ORDER BY
s.total_sales DESC) AS rnk
  FROM products p
  JOIN (
    SELECT product_id, SUM(quantity * price) AS total_sales
    FROM order_items
    GROUP BY product_id
  ) s ON p.product_id = s.product_id
) ranked
WHERE rnk <= 3;
```

Q193 Find customers who bought every single product in a given category.

```
SELECT o.customer_id
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
WHERE p.category_id = 5
GROUP BY o.customer_id
HAVING COUNT(DISTINCT p.product_id) = (
  SELECT COUNT(*) FROM products WHERE category_id = 5
);
```

Q194 Find consecutive groups of dates (islands) where a machine was continuously active.

```
SELECT machine_id, MIN(active_date) AS island_start, MAX(active_date) AS
island_end
FROM (
  SELECT machine_id, active_date,
         active_date - ROW_NUMBER() OVER (PARTITION BY machine_id ORDER BY
active_date)::INT AS grp
  FROM machine_activity
) t
GROUP BY machine_id, grp;
```

Q195 Find the longest consecutive run of login days for any single user.

```
SELECT user_id, COUNT(*) AS streak_length
FROM (
  SELECT user_id, login_date,
         login_date - ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY
login_date)::INT AS grp
  FROM logins
) t
GROUP BY user_id, grp
ORDER BY streak_length DESC
LIMIT 1;
```

09 Real Interview Favorites / Hard

CONTINUED · Q196–Q199

Q196 Transpose a table of exam scores so each subject becomes its own column per student.

```
SELECT student_id,
       MAX(CASE WHEN subject = 'Math' THEN score END) AS Math,
       MAX(CASE WHEN subject = 'Science' THEN score END) AS Science,
       MAX(CASE WHEN subject = 'English' THEN score END) AS English
FROM exam_scores
GROUP BY student_id;
```

Q197 Find employees who have an identical set of skills as another employee.

```
SELECT e1.employee_id, e2.employee_id AS matching_employee
FROM employee_skills e1
JOIN employee_skills e2
  ON e1.skill_id = e2.skill_id AND e1.employee_id <> e2.employee_id
GROUP BY e1.employee_id, e2.employee_id
HAVING COUNT(*) = (SELECT COUNT(*) FROM employee_skills WHERE employee_id
= e1.employee_id)
   AND COUNT(*) = (SELECT COUNT(*) FROM employee_skills WHERE employee_id
= e2.employee_id);
```

Q198 Find the three most recently registered customers in each country.

```
SELECT *
FROM (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY country ORDER BY signup_date
DESC) AS rnk
  FROM customers
) ranked
WHERE rnk <= 3;
```

Q199 Find products whose sales dropped for two consecutive months in a row.

```
SELECT product_id, month
FROM (
  SELECT product_id, month, total_sales,
         LAG(total_sales, 1) OVER (PARTITION BY product_id ORDER BY month)
AS prev1,
         LAG(total_sales, 2) OVER (PARTITION BY product_id ORDER BY month)
AS prev2
  FROM monthly_product_sales
) t
WHERE total_sales < prev1 AND prev1 < prev2;
```

09 Real Interview Favorites / Hard

CONTINUED · Q200–Q200

Q200 Find the employee-manager pair with the largest salary gap.

```
SELECT e.name AS employee, m.name AS manager, (m.salary - e.salary) AS
salary_gap
FROM employees e
JOIN employees m ON e.manager_id = m.id
ORDER BY salary_gap DESC
LIMIT 1;
```

10 Advanced Window Functions & Analytics

25 questions · Q201–Q225

Deeper OVER() patterns: weighted averages, centered windows, cohort ratios, and multi-period comparisons.

Q201 Calculate a weighted moving average of sales using quantity as the weight.

```
SELECT sale_date,
       SUM(amount * quantity) OVER (ORDER BY sale_date ROWS BETWEEN 2
PRECEDING AND CURRENT ROW)
       / NULLIF(SUM(quantity) OVER (ORDER BY sale_date ROWS BETWEEN 2
PRECEDING AND CURRENT ROW), 0)
       AS weighted_moving_avg
FROM sales;
```

Q202 Find the rank of each order within its own month by amount.

```
SELECT order_id, order_date, amount,
       RANK() OVER (PARTITION BY DATE_TRUNC('month', order_date) ORDER BY
amount DESC) AS month_rank
FROM orders;
```

Q203 Compute the difference between each order and the customer's first order amount.

```
SELECT customer_id, order_date, amount,
       amount - FIRST_VALUE(amount) OVER (
         PARTITION BY customer_id ORDER BY order_date
       ) AS diff_from_first
FROM orders;
```

Q204 Find a running total of revenue per product that resets at the start of each year.

```
SELECT product_id, sale_date, amount,
       SUM(amount) OVER (
         PARTITION BY product_id, EXTRACT(YEAR FROM sale_date)
         ORDER BY sale_date
       ) AS yearly_running_total
FROM product_sales;
```

10 Advanced Window Functions & Analytics

CONTINUED · Q205–Q208

Q205 Compute the ratio of each employee's salary to the maximum salary in their department.

```
SELECT name, department_id, salary,
       ROUND(salary::NUMERIC / MAX(salary) OVER (PARTITION BY
department_id), 2) AS pct_of_dept_max
FROM employees;
```

Q206 Find the second and third highest salaries in a single query using DENSE_RANK.

```
SELECT salary, rnk
FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rnk
  FROM employees
) ranked
WHERE rnk IN (2, 3);
```

Q207 Compute a centered 5-day moving average of daily sales (2 preceding, 2 following).

```
SELECT sale_date, amount,
       AVG(amount) OVER (
         ORDER BY sale_date
         ROWS BETWEEN 2 PRECEDING AND 2 FOLLOWING
       ) AS centered_avg
FROM sales;
```

Q208 Find the first order date on which a customer's running total exceeded 1000.

```
SELECT customer_id, order_date, running_total
FROM (
  SELECT customer_id, order_date,
         SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_date)
AS running_total,
         ROW_NUMBER() OVER (
           PARTITION BY customer_id
           ORDER BY order_date
         ) AS rn
  FROM orders
) t
WHERE running_total > 1000
QUALIFY ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date) =
1;
```

10 Advanced Window Functions & Analytics

CONTINUED · Q209–Q212

Q209 Compute a 2-period lag of stock prices for each symbol.

```
SELECT symbol, trade_date, price,
       LAG(price, 2) OVER (PARTITION BY symbol ORDER BY trade_date) AS
price_2_days_ago
FROM stock_prices;
```

Q210 Find each customer's spend-rank change between the current and previous month.

```
WITH monthly_rank AS (
  SELECT customer_id, month, SUM(amount) AS total,
         RANK() OVER (PARTITION BY month ORDER BY SUM(amount) DESC) AS rnk
  FROM monthly_customer_spend
  GROUP BY customer_id, month
)
SELECT customer_id, month, rnk,
       rnk - LAG(rnk) OVER (PARTITION BY customer_id ORDER BY month) AS
rank_change
FROM monthly_rank;
```

Q211 Compute the cumulative count of distinct products purchased by each customer over time.

```
SELECT customer_id, order_date, product_id,
       COUNT(DISTINCT product_id) OVER (
         PARTITION BY customer_id ORDER BY order_date
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS distinct_products_so_far
FROM order_items oi
JOIN orders o ON oi.order_id = o.order_id;
```

Q212 Find the employee in each department whose salary is closest to the department median.

```
WITH dept_median AS (
  SELECT department_id,
         PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary) AS
median_salary
  FROM employees
  GROUP BY department_id
)
SELECT e.name, e.department_id, e.salary
FROM employees e
JOIN dept_median d ON e.department_id = d.department_id
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY e.department_id ORDER BY ABS(e.salary - d.median_salary)
) = 1;
```

10 Advanced Window Functions & Analytics

CONTINUED · Q213–Q216

Q213 Compute the z-score of each employee's salary within the company.

```
SELECT name, salary,
       (salary - AVG(salary) OVER ()) / NULLIF(STDDEV(salary) OVER (), 0)
AS z_score
FROM employees;
```

Q214 Find the first and last order amount per customer using an unbounded window frame.

```
SELECT DISTINCT customer_id,
       FIRST_VALUE(amount) OVER (
         PARTITION BY customer_id ORDER BY order_date
         ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
       ) AS first_amount,
       LAST_VALUE(amount) OVER (
         PARTITION BY customer_id ORDER BY order_date
         ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
       ) AS last_amount
FROM orders;
```

Q215 Compute a 30-day trailing sum of daily revenue.

```
SELECT sale_date, amount,
       SUM(amount) OVER (
         ORDER BY sale_date
         RANGE BETWEEN INTERVAL '29 days' PRECEDING AND CURRENT ROW
       ) AS trailing_30day_sum
FROM daily_revenue;
```

Q216 Rank customers into deciles (10 buckets) based on lifetime spend.

```
SELECT customer_id, total_spend,
       NTILE(10) OVER (ORDER BY total_spend) AS spend_decile
FROM (
  SELECT customer_id, SUM(amount) AS total_spend
  FROM orders
  GROUP BY customer_id
) t;
```

10 Advanced Window Functions & Analytics

CONTINUED · Q217–Q220

Q217 Find the difference between each employee's salary and the next lower salary in the company.

```
SELECT name, salary,
       salary - LEAD(salary) OVER (ORDER BY salary DESC) AS
gap_to_next_lower
FROM employees;
```

Q218 Compute each department's share of total company headcount.

```
SELECT department_id, COUNT(*) AS headcount,
       ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 2) AS
pct_of_company
FROM employees
GROUP BY department_id;
```

Q219 Find products whose sales rank improved from Q1 to Q2.

```
WITH quarter_rank AS (
  SELECT product_id, quarter, total_sales,
         RANK() OVER (PARTITION BY quarter ORDER BY total_sales DESC) AS
rnk
  FROM quarterly_product_sales
)
SELECT q1.product_id, q1.rnk AS q1_rank, q2.rnk AS q2_rank
FROM quarter_rank q1
JOIN quarter_rank q2 ON q1.product_id = q2.product_id
WHERE q1.quarter = 'Q1' AND q2.quarter = 'Q2' AND q2.rnk < q1.rnk;
```

Q220 Compute a running average of order amount that excludes the current row.

```
SELECT customer_id, order_date, amount,
       AVG(amount) OVER (
         PARTITION BY customer_id ORDER BY order_date
         ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING
       ) AS avg_excluding_current
FROM orders;
```

10 Advanced Window Functions & Analytics

CONTINUED · Q221–Q224

Q221 Compare ROW_NUMBER, RANK, and DENSE_RANK side by side for the top earners.

```
SELECT name, salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num,
       RANK() OVER (ORDER BY salary DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rnk
FROM employees;
```

Q222 Compute, for each employee, the count of employees earning a higher salary (inverse rank).

```
SELECT name, salary,
       COUNT(*) OVER (ORDER BY salary DESC ROWS BETWEEN UNBOUNDED
PRECEDING AND 1 PRECEDING) AS higher_earners
FROM employees;
```

Q223 Find the largest single-day drop in stock price for each symbol using window functions.

```
SELECT symbol, trade_date,
       price - LAG(price) OVER (PARTITION BY symbol ORDER BY trade_date)
AS daily_change
FROM stock_prices
QUALIFY daily_change = MIN(price - LAG(price) OVER (PARTITION BY symbol
ORDER BY trade_date))
  OVER (PARTITION BY symbol);
```

Q224 Compute a 3-month rolling customer retention rate.

```
SELECT month,
       COUNT(DISTINCT CASE WHEN active_this_month AND active_last_month
THEN customer_id END)::NUMERIC
       / NULLIF(COUNT(DISTINCT CASE WHEN active_last_month THEN
customer_id END), 0) AS retention_rate
FROM customer_month_activity
GROUP BY month;
```

10 Advanced Window Functions & Analytics

CONTINUED · Q225–Q225

Q225 Find login gaps larger than 30 days between consecutive logins per user.

```
SELECT user_id, login_date,
       login_date - LAG(login_date) OVER (PARTITION BY user_id ORDER BY
login_date) AS gap_days
FROM logins
QUALIFY gap_days > 30;
```

11 Advanced Recursive CTEs & Hierarchies

20 questions · Q226–Q245

Multi-level trees, bill-of-materials explosions, path strings, and cycle-safe recursion.

Q226 Generate a series of dates between two dates using a recursive CTE.

```
WITH RECURSIVE date_series AS (
  SELECT DATE '2024-01-01' AS d
  UNION ALL
  SELECT d + 1 FROM date_series WHERE d + 1 <= DATE '2024-01-31'
)
SELECT d FROM date_series;
```

Q227 Build a materialized path string for each node in an org chart.

```
WITH RECURSIVE paths AS (
  SELECT id, name, manager_id, name::TEXT AS path
  FROM employees
  WHERE manager_id IS NULL
  UNION ALL
  SELECT e.id, e.name, e.manager_id, p.path || ' > ' || e.name
  FROM employees e
  JOIN paths p ON e.manager_id = p.id
)
SELECT * FROM paths ORDER BY path;
```

Q228 Explode a bill-of-materials to find all sub-components of a product.

```
WITH RECURSIVE bom AS (
  SELECT parent_product_id, component_id, quantity, 1 AS level
  FROM product_components
  WHERE parent_product_id = 500
  UNION ALL
  SELECT pc.parent_product_id, pc.component_id, pc.quantity, b.level + 1
  FROM product_components pc
  JOIN bom b ON pc.parent_product_id = b.component_id
)
SELECT * FROM bom ORDER BY level;
```

Q229 Detect a cycle in a manager-report relationship using a recursive CTE guard.

```
WITH RECURSIVE chain AS (
  SELECT id, manager_id, ARRAY[id] AS visited, FALSE AS is_cycle
  FROM employees
  UNION ALL
  SELECT e.id, c.manager_id, c.visited || e.id, e.id = ANY(c.visited)
  FROM employees e
  JOIN chain c ON e.manager_id = c.id
  WHERE NOT c.is_cycle
)
SELECT DISTINCT id FROM chain WHERE is_cycle;
```

11 Advanced Recursive CTEs & Hierarchies

CONTINUED · Q230–Q233

Q230 Find the total budget of a department including all of its child departments.

```
WITH RECURSIVE dept_tree AS (
  SELECT department_id, parent_department_id, budget
  FROM departments
  WHERE department_id = 10
  UNION ALL
  SELECT d.department_id, d.parent_department_id, d.budget
  FROM departments d
  JOIN dept_tree t ON d.parent_department_id = t.department_id
)
SELECT SUM(budget) AS total_budget FROM dept_tree;
```

Q231 Compute the number of direct reports at each level of the hierarchy.

```
WITH RECURSIVE levels AS (
  SELECT id, manager_id, 0 AS lvl
  FROM employees
  WHERE manager_id IS NULL
  UNION ALL
  SELECT e.id, e.manager_id, l.lvl + 1
  FROM employees e
  JOIN levels l ON e.manager_id = l.id
)
SELECT lvl, COUNT(*) AS employees_at_level
FROM levels
GROUP BY lvl
ORDER BY lvl;
```

Q232 Find the shortest reporting-chain path between two employees.

```
WITH RECURSIVE path AS (
  SELECT id, manager_id, ARRAY[id] AS chain, 0 AS steps
  FROM employees
  WHERE id = 55
  UNION ALL
  SELECT e.id, e.manager_id, p.chain || e.id, p.steps + 1
  FROM employees e
  JOIN path p ON e.id = p.manager_id
)
SELECT * FROM path WHERE id = 1 ORDER BY steps LIMIT 1;
```

Q233 Generate a numbered sequence from 1 to 100 without a numbers table.

```
WITH RECURSIVE seq AS (
  SELECT 1 AS n
  UNION ALL
  SELECT n + 1 FROM seq WHERE n < 100
)
SELECT n FROM seq;
```

11 Advanced Recursive CTEs & Hierarchies

CONTINUED · Q234–Q237

Q234 Find all categories and their nested sub-categories using a recursive CTE.

```
WITH RECURSIVE cat_tree AS (
  SELECT category_id, category_name, parent_category_id, 1 AS depth
  FROM categories
  WHERE parent_category_id IS NULL
  UNION ALL
  SELECT c.category_id, c.category_name, c.parent_category_id, ct.depth +
1
  FROM categories c
  JOIN cat_tree ct ON c.parent_category_id = ct.category_id
)
SELECT * FROM cat_tree ORDER BY depth, category_name;
```

Q235 Compute the total headcount under each director, including all indirect reports.

```
WITH RECURSIVE org AS (
  SELECT id, manager_id, id AS director_id
  FROM employees
  WHERE job_title = 'Director'
  UNION ALL
  SELECT e.id, e.manager_id, o.director_id
  FROM employees e
  JOIN org o ON e.manager_id = o.id
)
SELECT director_id, COUNT(*) AS total_headcount
FROM org
GROUP BY director_id;
```

Q236 Find the maximum depth of the organizational hierarchy.

```
WITH RECURSIVE depth AS (
  SELECT id, manager_id, 1 AS lvl
  FROM employees
  WHERE manager_id IS NULL
  UNION ALL
  SELECT e.id, e.manager_id, d.lvl + 1
  FROM employees e
  JOIN depth d ON e.manager_id = d.id
)
SELECT MAX(lvl) AS max_depth FROM depth;
```

Q237 Compute a running Fibonacci-style forecast sequence using a recursive CTE.

```
WITH RECURSIVE fib AS (
  SELECT 1 AS a, 1 AS b, 1 AS step
  UNION ALL
  SELECT b, a + b, step + 1 FROM fib WHERE step < 20
)
SELECT step, a AS fib_value FROM fib;
```

11 Advanced Recursive CTEs & Hierarchies

CONTINUED · Q238–Q241

Q238 Find all leaf-level employees (those with no direct reports).

```
WITH RECURSIVE tree AS (
  SELECT id FROM employees WHERE manager_id IS NULL
  UNION ALL
  SELECT e.id FROM employees e JOIN tree t ON e.manager_id = t.id
)
SELECT e.*
FROM employees e
WHERE NOT EXISTS (SELECT 1 FROM employees x WHERE x.manager_id = e.id);
```

Q239 Compute the total sales rolled up through a product category tree.

```
WITH RECURSIVE cat_tree AS (
  SELECT category_id, parent_category_id FROM categories
  WHERE category_id = 1
  UNION ALL
  SELECT c.category_id, c.parent_category_id
  FROM categories c
  JOIN cat_tree ct ON c.parent_category_id = ct.category_id
)
SELECT SUM(oi.quantity * oi.price) AS rolled_up_sales
FROM cat_tree ct
JOIN products p ON p.category_id = ct.category_id
JOIN order_items oi ON oi.product_id = p.product_id;
```

Q240 Find employees who are more than 3 levels deep in the reporting hierarchy.

```
WITH RECURSIVE hierarchy AS (
  SELECT id, manager_id, 0 AS depth
  FROM employees
  WHERE manager_id IS NULL
  UNION ALL
  SELECT e.id, e.manager_id, h.depth + 1
  FROM employees e
  JOIN hierarchy h ON e.manager_id = h.id
)
SELECT e.*
FROM employees e
JOIN hierarchy h ON e.id = h.id
WHERE h.depth > 3;
```

Q241 Traverse a friend-of-friend social graph up to 2 hops using recursion.

```
WITH RECURSIVE network AS (
  SELECT user_id, friend_id, 1 AS hops
  FROM friendships
  WHERE user_id = 1
  UNION ALL
  SELECT n.user_id, f.friend_id, n.hops + 1
  FROM friendships f
  JOIN network n ON f.user_id = n.friend_id
  WHERE n.hops < 2
)
SELECT DISTINCT friend_id FROM network;
```

11 Advanced Recursive CTEs & Hierarchies

CONTINUED · Q242–Q245

Q242 Find all ancestor categories of a given leaf category (walking upward).

```
WITH RECURSIVE ancestors AS (
  SELECT category_id, parent_category_id
  FROM categories
  WHERE category_id = 42
  UNION ALL
  SELECT c.category_id, c.parent_category_id
  FROM categories c
  JOIN ancestors a ON c.category_id = a.parent_category_id
)
SELECT * FROM ancestors;
```

Q243 Compute the cumulative project budget across a multi-level project/sub-project tree.

```
WITH RECURSIVE proj_tree AS (
  SELECT project_id, parent_project_id, budget
  FROM projects
  WHERE parent_project_id IS NULL
  UNION ALL
  SELECT p.project_id, p.parent_project_id, p.budget
  FROM projects p
  JOIN proj_tree t ON p.parent_project_id = t.project_id
)
SELECT SUM(budget) AS total_budget FROM proj_tree;
```

Q244 Find the reporting chain (top to bottom) as a single readable string for one employee.

```
WITH RECURSIVE chain AS (
  SELECT id, name, manager_id, name::TEXT AS path
  FROM employees
  WHERE id = 5
  UNION ALL
  SELECT e.id, e.name, e.manager_id, e.name || ' -> ' || c.path
  FROM employees e
  JOIN chain c ON e.id = c.manager_id
)
SELECT path FROM chain ORDER BY LENGTH(path) DESC LIMIT 1;
```

Q245 Flatten a threaded comments table into ordered conversation order using recursion.

```
WITH RECURSIVE thread AS (
  SELECT comment_id, parent_comment_id, comment_text, 1 AS depth,
         LPAD(comment_id::TEXT, 5, '0') AS sort_path
  FROM comments
  WHERE parent_comment_id IS NULL
  UNION ALL
  SELECT c.comment_id, c.parent_comment_id, c.comment_text, t.depth + 1,
         t.sort_path || '.' || LPAD(c.comment_id::TEXT, 5, '0')
  FROM comments c
  JOIN thread t ON c.parent_comment_id = t.comment_id
)
SELECT * FROM thread ORDER BY sort_path;
```

12 Complex JOIN Patterns

25 questions · Q246–Q270

Multi-key joins, anti-joins, lateral joins, self-joins, and set-based comparisons across tables.

Q246 Find customers who bought product A but never bought product B.

```
SELECT DISTINCT o.customer_id
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
WHERE oi.product_id = 'A'
  AND o.customer_id NOT IN (
    SELECT o2.customer_id
    FROM orders o2
    JOIN order_items oi2 ON o2.order_id = oi2.order_id
    WHERE oi2.product_id = 'B'
  );
```

Q247 Join two tables on multiple columns to find matching shipment records.

```
SELECT s.*
FROM shipments s
JOIN manifests m
  ON s.order_id = m.order_id
 AND s.warehouse_id = m.warehouse_id
 AND s.ship_date = m.ship_date;
```

Q248 Use a LATERAL join to get each customer's 3 most recent orders.

```
SELECT c.customer_id, c.name, recent.*
FROM customers c
CROSS JOIN LATERAL (
  SELECT order_id, order_date, amount
  FROM orders o
  WHERE o.customer_id = c.customer_id
  ORDER BY order_date DESC
  LIMIT 3
) recent;
```

Q249 Find employees and departments using a FULL OUTER JOIN to spot mismatches both ways.

```
SELECT e.name, d.department_name
FROM employees e
FULL OUTER JOIN departments d ON e.department_id = d.department_id
WHERE e.id IS NULL OR d.department_id IS NULL;
```

12 Complex JOIN Patterns

CONTINUED · Q250–Q253

Q250 Find pairs of orders from the same customer placed within 1 hour of each other.

```
SELECT o1.order_id AS order1, o2.order_id AS order2, o1.customer_id
FROM orders o1
JOIN orders o2
  ON o1.customer_id = o2.customer_id
 AND o1.order_id < o2.order_id
 AND ABS(EXTRACT(EPOCH FROM (o1.order_timestamp - o2.order_timestamp))) <=
3600;
```

Q251 Find products that appear in every warehouse's inventory (a relational-division join).

```
SELECT product_id
FROM inventory
GROUP BY product_id
HAVING COUNT(DISTINCT warehouse_id) = (SELECT COUNT(*) FROM warehouses);
```

Q252 Join a sales fact table to a date dimension to label each sale with its fiscal quarter.

```
SELECT s.sale_id, s.amount, d.fiscal_quarter, d.fiscal_year
FROM sales s
JOIN date_dim d ON s.sale_date = d.calendar_date;
```

Q253 Find customers whose shipping address differs from their billing address using a self-referencing join.

```
SELECT c.customer_id, a1.address AS billing, a2.address AS shipping
FROM customers c
JOIN addresses a1 ON c.customer_id = a1.customer_id AND a1.type =
'billing'
JOIN addresses a2 ON c.customer_id = a2.customer_id AND a2.type =
'shipping'
WHERE a1.address <> a2.address;
```

12 Complex JOIN Patterns

CONTINUED · Q254–Q257

Q254 Find the closest warehouse to each customer using a LATERAL join with distance calculation.

```
SELECT c.customer_id, nearest.warehouse_id, nearest.distance
FROM customers c
CROSS JOIN LATERAL (
  SELECT w.warehouse_id,
         SQRT(POWER(w.lat - c.lat, 2) + POWER(w.lon - c.lon, 2)) AS
distance
  FROM warehouses w
  ORDER BY distance ASC
  LIMIT 1
) nearest;
```

Q255 Find employees assigned to a project but missing from the timesheets table (anti-join).

```
SELECT pa.employee_id, pa.project_id
FROM project_assignments pa
WHERE NOT EXISTS (
  SELECT 1 FROM timesheets t
  WHERE t.employee_id = pa.employee_id AND t.project_id = pa.project_id
);
```

Q256 Join three tables to compute each salesperson's commission for the month.

```
SELECT r.rep_name, SUM(o.amount) AS total_sales, SUM(o.amount) *
c.commission_rate AS commission
FROM sales_reps r
JOIN orders o ON r.rep_id = o.rep_id
JOIN commission_rates c ON r.rep_id = c.rep_id
WHERE DATE_TRUNC('month', o.order_date) = DATE_TRUNC('month',
CURRENT_DATE)
GROUP BY r.rep_name, c.commission_rate;
```

Q257 Find customers who have orders in EVERY month of the current year (relational division).

```
SELECT customer_id
FROM orders
WHERE EXTRACT(YEAR FROM order_date) = EXTRACT(YEAR FROM CURRENT_DATE)
GROUP BY customer_id
HAVING COUNT(DISTINCT EXTRACT(MONTH FROM order_date)) = 12;
```

12 Complex JOIN Patterns

CONTINUED · Q258–Q261

Q258 Find overlapping date-range bookings for the same meeting room using a range join.

```
SELECT b1.booking_id AS booking1, b2.booking_id AS booking2
FROM bookings b1
JOIN bookings b2
  ON b1.room_id = b2.room_id
 AND b1.booking_id < b2.booking_id
 AND b1.start_time < b2.end_time
 AND b2.start_time < b1.end_time;
```

Q259 Join employees to their most recent performance review using a LATERAL subquery.

```
SELECT e.name, r.review_date, r.rating
FROM employees e
CROSS JOIN LATERAL (
  SELECT review_date, rating
  FROM performance_reviews pr
  WHERE pr.employee_id = e.id
  ORDER BY review_date DESC
  LIMIT 1
) r;
```

Q260 Find products sold in one store but not in another, using an anti-join on a multi-column key.

```
SELECT p.product_id
FROM store_inventory p
WHERE p.store_id = 'A'
  AND NOT EXISTS (
    SELECT 1 FROM store_inventory q
    WHERE q.store_id = 'B' AND q.product_id = p.product_id
  );
```

Q261 Join a transactions table to itself to find refund transactions matching an original sale.

```
SELECT t1.transaction_id AS sale_id, t2.transaction_id AS refund_id
FROM transactions t1
JOIN transactions t2
  ON t1.order_id = t2.order_id
 AND t1.type = 'sale' AND t2.type = 'refund'
 AND t1.amount = -t2.amount;
```

12 Complex JOIN Patterns

CONTINUED · Q262–Q265

Q262 Find employees working on projects outside their own department using a multi-table join.

```
SELECT e.name, pr.project_name, pr.department_id AS project_department
FROM employees e
JOIN project_assignments pa ON e.id = pa.employee_id
JOIN projects pr ON pa.project_id = pr.project_id
WHERE e.department_id <> pr.department_id;
```

Q263 Find the best-selling product in each store using a LATERAL join with TOP-1 semantics.

```
SELECT s.store_id, top_product.product_id, top_product.total_qty
FROM stores s
CROSS JOIN LATERAL (
  SELECT product_id, SUM(quantity) AS total_qty
  FROM store_sales ss
  WHERE ss.store_id = s.store_id
  GROUP BY product_id
  ORDER BY total_qty DESC
  LIMIT 1
) top_product;
```

Q264 Find pairs of products frequently bought together (market-basket self-join).

```
SELECT oi1.product_id AS product_a, oi2.product_id AS product_b, COUNT(*)
AS times_together
FROM order_items oi1
JOIN order_items oi2
  ON oi1.order_id = oi2.order_id AND oi1.product_id < oi2.product_id
GROUP BY oi1.product_id, oi2.product_id
ORDER BY times_together DESC;
```

Q265 Join employees with their department and a lookup of the department's building floor.

```
SELECT e.name, d.department_name, f.floor_number
FROM employees e
JOIN departments d ON e.department_id = d.department_id
JOIN building_floors f ON d.location = f.location;
```

12 Complex JOIN Patterns

CONTINUED · Q266–Q269

Q266 Find customers whose order history exactly matches another customer's order history (basket equality).

```
SELECT o1.customer_id, o2.customer_id AS matching_customer
FROM (SELECT DISTINCT customer_id, product_id FROM order_items oi JOIN
orders o ON oi.order_id=o.order_id) o1
JOIN (SELECT DISTINCT customer_id, product_id FROM order_items oi JOIN
orders o ON oi.order_id=o.order_id) o2
  ON o1.product_id = o2.product_id AND o1.customer_id < o2.customer_id
GROUP BY o1.customer_id, o2.customer_id
HAVING COUNT(*) = (SELECT COUNT(*) FROM order_items oi JOIN orders o ON
oi.order_id=o.order_id WHERE o.customer_id = o1.customer_id)
   AND COUNT(*) = (SELECT COUNT(*) FROM order_items oi JOIN orders o ON
oi.order_id=o.order_id WHERE o.customer_id = o2.customer_id);
```

Q267 Find managers who share at least one direct report's department with another manager (self-join with grouping).

```
SELECT DISTINCT m1.id AS manager1, m2.id AS manager2, e1.department_id
FROM employees e1
JOIN employees m1 ON e1.manager_id = m1.id
JOIN employees e2 ON e1.department_id = e2.department_id AND e1.manager_id
<> e2.manager_id
JOIN employees m2 ON e2.manager_id = m2.id
WHERE m1.id < m2.id;
```

Q268 Join orders to a currency-rates table to convert every order amount to USD as of the order date.

```
SELECT o.order_id, o.amount, o.currency, o.amount * r.usd_rate AS
amount_usd
FROM orders o
JOIN currency_rates r
  ON o.currency = r.currency AND o.order_date = r.rate_date;
```

Q269 Find employees who share the exact same set of project assignments as another employee, using a join-based set comparison.

```
SELECT pa1.employee_id AS employee_a, pa2.employee_id AS employee_b
FROM project_assignments pa1
JOIN project_assignments pa2
  ON pa1.project_id = pa2.project_id AND pa1.employee_id < pa2.employee_id
GROUP BY pa1.employee_id, pa2.employee_id
HAVING COUNT(*) = (SELECT COUNT(*) FROM project_assignments WHERE
employee_id = pa1.employee_id)
   AND COUNT(*) = (SELECT COUNT(*) FROM project_assignments WHERE
employee_id = pa2.employee_id);
```

12 Complex JOIN Patterns

CONTINUED · Q270–Q270

Q270 Join a returns table back to the original order_items to compute the net quantity sold per product.

```
SELECT oi.product_id,
       SUM(oi.quantity) - COALESCE(SUM(r.quantity_returned), 0) AS
net_quantity_sold
FROM order_items oi
LEFT JOIN returns r
  ON oi.order_id = r.order_id AND oi.product_id = r.product_id
GROUP BY oi.product_id;
```

13 Advanced Aggregation Pipelines

20 questions · Q271–Q290

Multi-stage GROUP BY pipelines, conditional aggregation, rollups, and cube-style summaries.

Q271 Build a sales summary with subtotals per region and a grand total using ROLLUP.

```
SELECT region, product_category, SUM(amount) AS total_sales
FROM sales
GROUP BY ROLLUP (region, product_category);
```

Q272 Build a cross-tabulation of headcount by department and job title using CUBE.

```
SELECT department_id, job_title, COUNT(*) AS headcount
FROM employees
GROUP BY CUBE (department_id, job_title);
```

Q273 Compute conditional aggregates: total revenue split by online vs in-store channel in one row per month.

```
SELECT DATE_TRUNC('month', sale_date) AS month,
       SUM(CASE WHEN channel = 'online' THEN amount ELSE 0 END) AS
online_revenue,
       SUM(CASE WHEN channel = 'in_store' THEN amount ELSE 0 END) AS
in_store_revenue
FROM sales
GROUP BY DATE_TRUNC('month', sale_date);
```

Q274 Find the funnel conversion rate from signup to first purchase using staged aggregation.

```
WITH signups AS (SELECT COUNT(*) AS total_signups FROM customers),
     purchasers AS (SELECT COUNT(DISTINCT customer_id) AS total_purchasers
FROM orders)
SELECT p.total_purchasers, s.total_signups,
       ROUND(100.0 * p.total_purchasers / s.total_signups, 2) AS
conversion_rate
FROM signups s, purchasers p;
```

13 Advanced Aggregation Pipelines

CONTINUED · Q275–Q278

Q275 Compute a multi-level aggregation pipeline: daily totals, then monthly averages of those daily totals.

```
WITH daily AS (
  SELECT sale_date, SUM(amount) AS daily_total
  FROM sales
  GROUP BY sale_date
)
SELECT DATE_TRUNC('month', sale_date) AS month, AVG(daily_total) AS
avg_daily_sales
FROM daily
GROUP BY DATE_TRUNC('month', sale_date);
```

Q276 Find the top 3 revenue-generating categories, with all others rolled into an 'Other' bucket.

```
WITH cat_totals AS (
  SELECT category_name, SUM(amount) AS total,
         RANK() OVER (ORDER BY SUM(amount) DESC) AS rnk
  FROM sales s JOIN categories c ON s.category_id = c.category_id
  GROUP BY category_name
)
SELECT CASE WHEN rnk <= 3 THEN category_name ELSE 'Other' END AS category,
SUM(total) AS total
FROM cat_totals
GROUP BY CASE WHEN rnk <= 3 THEN category_name ELSE 'Other' END;
```

Q277 Compute distinct customer counts per channel using FILTER-style conditional counting.

```
SELECT COUNT(*) FILTER (WHERE channel = 'online') AS online_customers,
       COUNT(*) FILTER (WHERE channel = 'in_store') AS in_store_customers
FROM (SELECT DISTINCT customer_id, channel FROM orders) t;
```

Q278 Build a summary table showing min, max, avg and stddev of order amount per customer segment.

```
SELECT segment,
       MIN(amount) AS min_amount,
       MAX(amount) AS max_amount,
       ROUND(AVG(amount), 2) AS avg_amount,
       ROUND(STDDEV(amount), 2) AS stddev_amount
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY segment;
```

13 Advanced Aggregation Pipelines

CONTINUED · Q279–Q282

Q279 Compute year-over-year comparison of quarterly revenue in a single pivoted result set.

```
SELECT quarter,
       SUM(CASE WHEN year = 2024 THEN revenue END) AS revenue_2024,
       SUM(CASE WHEN year = 2025 THEN revenue END) AS revenue_2025
FROM quarterly_revenue
GROUP BY quarter
ORDER BY quarter;
```

Q280 Find the average basket size (items per order) segmented by customer loyalty tier.

```
SELECT c.loyalty_tier, AVG(item_count) AS avg_basket_size
FROM (
  SELECT order_id, customer_id, COUNT(*) AS item_count
  FROM order_items oi JOIN orders o ON oi.order_id = o.order_id
  GROUP BY order_id, customer_id
) basket
JOIN customers c ON basket.customer_id = c.customer_id
GROUP BY c.loyalty_tier;
```

Q281 Compute a two-stage aggregation: total per employee per project, then average per department.

```
WITH emp_project_hours AS (
  SELECT e.department_id, t.employee_id, SUM(t.hours) AS total_hours
  FROM timesheets t
  JOIN employees e ON t.employee_id = e.id
  GROUP BY e.department_id, t.employee_id
)
SELECT department_id, AVG(total_hours) AS avg_hours_per_employee
FROM emp_project_hours
GROUP BY department_id;
```

Q282 Find the percentage of total revenue contributed by each sales channel using window and group aggregation together.

```
SELECT channel, SUM(amount) AS channel_revenue,
       ROUND(100.0 * SUM(amount) / SUM(SUM(amount)) OVER (), 2) AS
pct_of_total
FROM sales
GROUP BY channel;
```

13 Advanced Aggregation Pipelines

CONTINUED · Q283–Q286

Q283 Build a GROUPING SETS report combining totals by region alone, by product alone, and by both.

```
SELECT region, product_category, SUM(amount) AS total_sales
FROM sales
GROUP BY GROUPING SETS ((region), (product_category), (region,
product_category));
```

Q284 Compute a customer cohort retention matrix (signup month vs activity month).

```
SELECT DATE_TRUNC('month', c.signup_date) AS cohort_month,
       DATE_TRUNC('month', o.order_date) AS activity_month,
       COUNT(DISTINCT o.customer_id) AS active_customers
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY DATE_TRUNC('month', c.signup_date), DATE_TRUNC('month',
o.order_date)
ORDER BY cohort_month, activity_month;
```

Q285 Find the incremental revenue contribution of each new product launched this year versus last year's baseline.

```
SELECT p.product_id,
       SUM(CASE WHEN EXTRACT(YEAR FROM o.order_date) = EXTRACT(YEAR FROM
CURRENT_DATE) THEN oi.quantity*oi.price ELSE 0 END) AS this_year,
       SUM(CASE WHEN EXTRACT(YEAR FROM o.order_date) = EXTRACT(YEAR FROM
CURRENT_DATE) - 1 THEN oi.quantity*oi.price ELSE 0 END) AS last_year
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
JOIN orders o ON oi.order_id = o.order_id
GROUP BY p.product_id;
```

Q286 Compute an aging report bucketing outstanding invoices into 0-30, 31-60, 61-90 and 90+ days.

```
SELECT
  CASE
    WHEN CURRENT_DATE - due_date <= 30 THEN '0-30'
    WHEN CURRENT_DATE - due_date <= 60 THEN '31-60'
    WHEN CURRENT_DATE - due_date <= 90 THEN '61-90'
    ELSE '90+'
  END AS aging_bucket,
  SUM(amount_due) AS total_due
FROM invoices
WHERE paid = FALSE
GROUP BY 1;
```

13 Advanced Aggregation Pipelines

CONTINUED · Q287–Q290

Q287 Compute the percentage of orders that are first-time versus repeat purchases, per month.

```
WITH first_orders AS (
  SELECT customer_id, MIN(order_date) AS first_order_date
  FROM orders
  GROUP BY customer_id
)
SELECT DATE_TRUNC('month', o.order_date) AS month,
       COUNT(*) FILTER (WHERE o.order_date = f.first_order_date) AS
first_time_orders,
       COUNT(*) FILTER (WHERE o.order_date <> f.first_order_date) AS
repeat_orders
FROM orders o
JOIN first_orders f ON o.customer_id = f.customer_id
GROUP BY DATE_TRUNC('month', o.order_date);
```

Q288 Build a summary of average discount percentage applied, grouped by product category and customer tier.

```
SELECT p.category_id, c.loyalty_tier,
       ROUND(AVG(100.0 * (p.price - oi.price) / NULLIF(p.price, 0)), 2) AS
avg_discount_pct
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
JOIN orders o ON oi.order_id = o.order_id
JOIN customers c ON o.customer_id = c.customer_id
GROUP BY p.category_id, c.loyalty_tier;
```

Q289 Compute total, average, and median order value together in a single aggregation query.

```
SELECT COUNT(*) AS total_orders,
       SUM(amount) AS total_revenue,
       ROUND(AVG(amount), 2) AS avg_order_value,
       PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY amount) AS
median_order_value
FROM orders;
```

Q290 Find the percentage of employees in each department who received a raise in the last 12 months.

```
SELECT e.department_id,
       ROUND(100.0 * COUNT(DISTINCT sh.employee_id) / COUNT(DISTINCT
e.id), 2) AS pct_received_raise
FROM employees e
LEFT JOIN salary_history sh
  ON e.id = sh.employee_id AND sh.raise_date >= CURRENT_DATE - INTERVAL
'12 months'
GROUP BY e.department_id;
```

14 Schema Modifications & DDL

25 questions · Q291–Q315

ALTER, CREATE, constraints, indexes, views, and other data-definition tasks a senior engineer is expected to know.

Q291 Create a new table for storing customer addresses with proper primary and foreign keys.

```
CREATE TABLE addresses (
  address_id SERIAL PRIMARY KEY,
  customer_id INT NOT NULL REFERENCES customers(customer_id),
  address_line VARCHAR(255) NOT NULL,
  city VARCHAR(100),
  country VARCHAR(100),
  type VARCHAR(20) CHECK (type IN ('billing','shipping'))
);
```

Q292 Add a new NOT NULL column with a default value to an existing large table.

```
ALTER TABLE employees
ADD COLUMN is_active BOOLEAN NOT NULL DEFAULT TRUE;
```

Q293 Rename a column without breaking existing views that reference it.

```
ALTER TABLE customers
RENAME COLUMN phone TO phone_number;
```

Q294 Add a foreign key constraint to an existing column after cleaning up orphaned rows.

```
DELETE FROM orders
WHERE customer_id NOT IN (SELECT customer_id FROM customers);

ALTER TABLE orders
ADD CONSTRAINT fk_orders_customer
FOREIGN KEY (customer_id) REFERENCES customers(customer_id);
```

14 Schema Modifications & DDL

CONTINUED · Q295–Q298

Q295 Create a composite index to speed up a common two-column filter.

```
CREATE INDEX idx_orders_customer_date
ON orders (customer_id, order_date);
```

Q296 Create a partial index that only covers active (non-deleted) rows.

```
CREATE INDEX idx_active_employees
ON employees (department_id)
WHERE is_active = TRUE;
```

Q297 Add a CHECK constraint to enforce that salary is always positive.

```
ALTER TABLE employees
ADD CONSTRAINT chk_salary_positive CHECK (salary > 0);
```

Q298 Create a view that exposes only non-sensitive employee columns.

```
CREATE VIEW employee_directory AS
SELECT id, name, department_id, job_title, email
FROM employees;
```

14 Schema Modifications & DDL

CONTINUED · Q299–Q302

Q299 Create a materialized view for a slow-to-compute monthly sales summary and refresh it.

```
CREATE MATERIALIZED VIEW monthly_sales_summary AS
SELECT DATE_TRUNC('month', sale_date) AS month, SUM(amount) AS total_sales
FROM sales
GROUP BY DATE_TRUNC('month', sale_date);

REFRESH MATERIALIZED VIEW monthly_sales_summary;
```

Q300 Alter a column's data type from VARCHAR(50) to VARCHAR(255).

```
ALTER TABLE products
ALTER COLUMN product_name TYPE VARCHAR(255);
```

Q301 Add a unique constraint on a column that should never have duplicate values.

```
ALTER TABLE employees
ADD CONSTRAINT uq_employee_email UNIQUE (email);
```

Q302 Drop a column that is no longer used by the application.

```
ALTER TABLE products
DROP COLUMN legacy_sku;
```

14 Schema Modifications & DDL

CONTINUED · Q303–Q306

Q303 Create a table partitioned by range on the order_date column.

```
CREATE TABLE orders (
  order_id SERIAL,
  customer_id INT,
  order_date DATE NOT NULL,
  amount DECIMAL(10,2)
) PARTITION BY RANGE (order_date);

CREATE TABLE orders_2025 PARTITION OF orders
FOR VALUES FROM ('2025-01-01') TO ('2026-01-01');
```

Q304 Create a sequence and use it to backfill an existing ID column.

```
CREATE SEQUENCE invoice_seq START 10000;

UPDATE invoices
SET invoice_number = nextval('invoice_seq')
WHERE invoice_number IS NULL;
```

Q305 Add a self-referencing foreign key to support a manager hierarchy.

```
ALTER TABLE employees
ADD CONSTRAINT fk_manager
FOREIGN KEY (manager_id) REFERENCES employees(id);
```

Q306 Create a table with a generated (computed) column for total line-item value.

```
CREATE TABLE order_items (
  order_id INT,
  product_id INT,
  quantity INT,
  price DECIMAL(10,2),
  line_total DECIMAL(10,2) GENERATED ALWAYS AS (quantity * price) STORED
);
```

14 Schema Modifications & DDL

CONTINUED · Q307–Q310

Q307 Rename a table safely while keeping a backward-compatible view under the old name.

```
ALTER TABLE clients RENAME TO customers;

CREATE VIEW clients AS SELECT * FROM customers;
```

Q308 Add an ON DELETE CASCADE behavior to an existing foreign key.

```
ALTER TABLE order_items
DROP CONSTRAINT fk_order_items_order,
ADD CONSTRAINT fk_order_items_order
FOREIGN KEY (order_id) REFERENCES orders(order_id) ON DELETE CASCADE;
```

Q309 Create a lookup table for status codes and migrate an existing free-text status column to reference it.

```
CREATE TABLE order_statuses (
  status_id SERIAL PRIMARY KEY,
  status_name VARCHAR(30) UNIQUE NOT NULL
);

INSERT INTO order_statuses (status_name)
SELECT DISTINCT status FROM orders;

ALTER TABLE orders ADD COLUMN status_id INT REFERENCES
order_statuses(status_id);

UPDATE orders o
SET status_id = s.status_id
FROM order_statuses s
WHERE o.status = s.status_name;
```

Q310 Create a trigger that automatically updates an updated_at timestamp on row modification.

```
CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
  NEW.updated_at = NOW();
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_customers_updated_at
BEFORE UPDATE ON customers
FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

14 Schema Modifications & DDL

CONTINUED · Q311–Q314

Q311 Add a composite primary key to a many-to-many junction table.

```
ALTER TABLE enrollments
ADD CONSTRAINT pk_enrollments PRIMARY KEY (student_id, course_id);
```

Q312 Create a schema and move a set of reporting tables into it.

```
CREATE SCHEMA reporting;

ALTER TABLE monthly_sales_summary SET SCHEMA reporting;
```

Q313 Add a default expression that auto-populates a created_at column with the current timestamp.

```
ALTER TABLE orders
ADD COLUMN created_at TIMESTAMP NOT NULL DEFAULT NOW();
```

Q314 Safely drop a table only if no foreign keys reference it.

```
DROP TABLE IF EXISTS temp_staging_orders;
```

14 Schema Modifications & DDL

CONTINUED · Q315–Q315

Q315 Add a table-level exclusion constraint to prevent overlapping room bookings.

```
ALTER TABLE bookings
ADD CONSTRAINT no_overlapping_bookings
EXCLUDE USING gist (
  room_id WITH =,
  tsrange(start_time, end_time) WITH &&
);
```

15 Performance Tuning & Query Optimization

20 questions · Q316–Q335

Reading execution plans, indexing strategy, and rewriting slow queries for large tables.

Q316 Use EXPLAIN ANALYZE to inspect how a slow customer-order query is being executed.

```
EXPLAIN ANALYZE
SELECT c.name, SUM(o.amount)
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.name;
```

Q317 Rewrite a NOT IN subquery as a NOT EXISTS to avoid NULL-related performance traps.

```
-- Slow / risky with NULLs:
-- SELECT * FROM customers WHERE customer_id NOT IN (SELECT customer_id
FROM orders);

-- Faster and NULL-safe:
SELECT c.*
FROM customers c
WHERE NOT EXISTS (
  SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
);
```

Q318 Rewrite a query that applies a function to an indexed column so the index can still be used.

```
-- Prevents index usage:
-- SELECT * FROM orders WHERE EXTRACT(YEAR FROM order_date) = 2025;

-- Index-friendly range scan instead:
SELECT *
FROM orders
WHERE order_date >= '2025-01-01' AND order_date < '2026-01-01';
```

Q319 Add a covering index so a common SELECT can be satisfied entirely from the index.

```
CREATE INDEX idx_orders_covering
ON orders (customer_id, order_date) INCLUDE (amount);
```

Performance Tuning & Query Optimization

CONTINUED · Q320–Q323

Q320 Rewrite a correlated subquery used for existence checks into a JOIN for large tables.

```
-- Correlated subquery (can be slow row-by-row):
-- SELECT * FROM employees e WHERE EXISTS (SELECT 1 FROM
project_assignments p WHERE p.employee_id = e.id);

-- Semi-join rewrite:
SELECT DISTINCT e.*
FROM employees e
JOIN project_assignments p ON p.employee_id = e.id;
```

Q321 Use a CTE with MATERIALIZED hint to avoid re-computing an expensive aggregation multiple times.

```
WITH monthly_totals AS MATERIALIZED (
  SELECT DATE_TRUNC('month', sale_date) AS month, SUM(amount) AS total
  FROM sales
  GROUP BY DATE_TRUNC('month', sale_date)
)
SELECT * FROM monthly_totals WHERE total > 100000
UNION ALL
SELECT * FROM monthly_totals WHERE total < 1000;
```

Q322 Batch a large DELETE into smaller chunks to avoid long lock times.

```
DELETE FROM logs
WHERE ctid IN (
  SELECT ctid FROM logs
  WHERE log_date < CURRENT_DATE - INTERVAL '1 year'
  LIMIT 10000
);
-- Repeat in a loop until zero rows are affected.
```

Q323 Identify unused indexes that can be dropped to speed up writes.

```
SELECT relname AS table_name, indexrelname AS index_name, idx_scan
FROM pg_stat_user_indexes
WHERE idx_scan = 0
ORDER BY relname;
```

15 Performance Tuning & Query Optimization

CONTINUED · Q324–Q327

Q324 Rewrite a query using multiple OR conditions on the same column as an IN list for a cleaner plan.

```
-- Harder to optimize:
-- SELECT * FROM orders WHERE status = 'pending' OR status = 'processing'
OR status = 'shipped';

-- Cleaner and equally indexable:
SELECT *
FROM orders
WHERE status IN ('pending', 'processing', 'shipped');
```

Q325 Use a window function instead of a self-join to avoid a quadratic execution plan.

```
-- Slower self-join approach:
-- SELECT a.*, (SELECT MAX(amount) FROM orders b WHERE b.customer_id =
a.customer_id) FROM orders a;

-- Single-pass window function instead:
SELECT customer_id, amount,
       MAX(amount) OVER (PARTITION BY customer_id) AS max_amount
FROM orders;
```

Q326 Estimate table and index bloat to decide when a VACUUM/REINDEX is needed.

```
SELECT relname, n_dead_tup, n_live_tup,
       ROUND(100.0 * n_dead_tup / NULLIF(n_live_tup + n_dead_tup, 0), 2)
AS dead_pct
FROM pg_stat_user_tables
ORDER BY dead_pct DESC;
```

Q327 Analyze a query plan to confirm a sequential scan is being replaced by an index scan after adding an index.

```
CREATE INDEX idx_customers_email ON customers (email);

EXPLAIN ANALYZE
SELECT * FROM customers WHERE email = 'alice@example.com';
```

15 Performance Tuning & Query Optimization

CONTINUED · Q328–Q331

Q328 Rewrite a SELECT * query to only fetch needed columns for a high-traffic endpoint.

```
-- Avoid over-fetching:
-- SELECT * FROM products WHERE category_id = 3;

-- Fetch only what the endpoint needs:
SELECT product_id, product_name, price
FROM products
WHERE category_id = 3;
```

Q329 Use LIMIT with an indexed ORDER BY to efficiently paginate a large orders table (keyset pagination).

```
SELECT order_id, order_date, amount
FROM orders
WHERE order_date < '2025-06-01'
ORDER BY order_date DESC
LIMIT 20;
```

Q330 Use EXPLAIN to compare a query before and after adding a partial index for frequently filtered active rows.

```
CREATE INDEX idx_active_orders ON orders (order_date) WHERE status =
'active';

EXPLAIN ANALYZE
SELECT * FROM orders WHERE status = 'active' ORDER BY order_date DESC
LIMIT 50;
```

Q331 Break a single massive multi-join query into staged temp tables for readability and speed.

```
CREATE TEMP TABLE tmp_customer_orders AS
SELECT customer_id, SUM(amount) AS total_spend
FROM orders
GROUP BY customer_id;

SELECT c.name, t.total_spend
FROM customers c
JOIN tmp_customer_orders t ON c.customer_id = t.customer_id
WHERE t.total_spend > 5000;
```

15 Performance Tuning & Query Optimization

CONTINUED · Q332–Q335

Q332 Use ANALYZE to refresh table statistics after a large bulk load so the planner picks good plans.

```
COPY orders FROM '/data/orders_bulk.csv' WITH (FORMAT csv, HEADER true);

ANALYZE orders;
```

Q333 Detect and resolve a query that is blocked waiting on a lock from another session.

```
SELECT blocked.pid AS blocked_pid, blocking.pid AS blocking_pid,
blocked.query AS blocked_query
FROM pg_stat_activity blocked
JOIN pg_locks bl ON blocked.pid = bl.pid AND NOT bl.granted
JOIN pg_locks kl ON bl.locktype = kl.locktype AND bl.relation =
kl.relation AND kl.granted
JOIN pg_stat_activity blocking ON kl.pid = blocking.pid
WHERE blocked.pid <> blocking.pid;
```

Q334 Rewrite a query that uses DISTINCT to mask a join fan-out bug, replacing it with a properly pre-aggregated subquery.

```
-- Masking symptom, not the cause:
-- SELECT DISTINCT c.customer_id, c.name FROM customers c JOIN orders o ON
c.customer_id = o.customer_id;

-- Pre-aggregate before joining to avoid the fan-out entirely:
SELECT c.customer_id, c.name, o.order_count
FROM customers c
JOIN (
  SELECT customer_id, COUNT(*) AS order_count
  FROM orders
  GROUP BY customer_id
) o ON c.customer_id = o.customer_id;
```

Q335 Use a hash-friendly equality rewrite to replace a slow range-based join condition where possible.

```
-- Range join can be costly at scale:
-- SELECT * FROM events e JOIN sessions s ON e.event_time BETWEEN
s.start_time AND s.end_time;

-- Pre-bucket into a shared key when granularity allows an equality join:
SELECT *
FROM events e
JOIN sessions s ON DATE_TRUNC('hour', e.event_time) = DATE_TRUNC('hour',
s.start_time);
```

16 Senior-Level Interview Challenges

15 questions · Q336–Q350

Capstone problems that combine window functions, recursion, joins and aggregation in a single query.

Q336 Find the top 2 highest-paid employees per department, tie-broken by earliest join date, and show the department average alongside.

```
SELECT *
FROM (
  SELECT e.*, d.avg_salary,
         ROW_NUMBER() OVER (
           PARTITION BY e.department_id
           ORDER BY e.salary DESC, e.join_date ASC
         ) AS rnk
  FROM employees e
  JOIN (
    SELECT department_id, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department_id
  ) d ON e.department_id = d.department_id
) ranked
WHERE rnk <= 2;
```

Q337 Build a full customer RFM (Recency, Frequency, Monetary) segmentation in one query.

```
WITH rfm AS (
  SELECT customer_id,
         CURRENT_DATE - MAX(order_date) AS recency_days,
         COUNT(*) AS frequency,
         SUM(amount) AS monetary
  FROM orders
  GROUP BY customer_id
),
scored AS (
  SELECT customer_id,
         NTILE(5) OVER (ORDER BY recency_days ASC) AS r_score,
         NTILE(5) OVER (ORDER BY frequency DESC) AS f_score,
         NTILE(5) OVER (ORDER BY monetary DESC) AS m_score
  FROM rfm
)
SELECT *, (r_score + f_score + m_score) AS rfm_total
FROM scored
ORDER BY rfm_total DESC;
```

Q338 Find, for each department, the longest gap in months between any two consecutive employee hires.

```
WITH hire_gaps AS (
  SELECT department_id, join_date,
         join_date - LAG(join_date) OVER (PARTITION BY department_id ORDER
BY join_date) AS gap
  FROM employees
)
SELECT department_id, MAX(gap) AS longest_hiring_gap
FROM hire_gaps
GROUP BY department_id;
```

Q339 Combine a recursive CTE and window function to rank each employee's salary among their entire management chain.

```
WITH RECURSIVE chain AS (
  SELECT id, name, manager_id, salary, id AS root
  FROM employees
  WHERE manager_id IS NULL
  UNION ALL
  SELECT e.id, e.name, e.manager_id, e.salary, c.root
  FROM employees e
  JOIN chain c ON e.manager_id = c.id
)
SELECT id, name, root, salary,
       RANK() OVER (PARTITION BY root ORDER BY salary DESC) AS
rank_in_chain
FROM chain;
```

Q340 Find customers whose order frequency in the last 90 days has doubled compared to the previous 90 days.

```
WITH periods AS (
  SELECT customer_id,
         COUNT(*) FILTER (WHERE order_date >= CURRENT_DATE - INTERVAL '90
days') AS recent_count,
         COUNT(*) FILTER (WHERE order_date >= CURRENT_DATE - INTERVAL '180
days'
                            AND order_date < CURRENT_DATE - INTERVAL '90
days') AS prior_count
  FROM orders
  GROUP BY customer_id
)
SELECT customer_id, recent_count, prior_count
FROM periods
WHERE prior_count > 0 AND recent_count >= 2 * prior_count;
```

Q341 Compute a full gaps-and-islands report of active subscription periods per user, merging overlapping ranges.

```
WITH ordered AS (
  SELECT user_id, start_date, end_date,
         MAX(end_date) OVER (
           PARTITION BY user_id ORDER BY start_date
           ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING
         ) AS prev_max_end
  FROM subscriptions
),
flagged AS (
  SELECT *, CASE WHEN start_date > prev_max_end OR prev_max_end IS NULL
THEN 1 ELSE 0 END AS is_new_island
  FROM ordered
),
grouped AS (
  SELECT *, SUM(is_new_island) OVER (PARTITION BY user_id ORDER BY
start_date) AS island_id
  FROM flagged
)
SELECT user_id, island_id, MIN(start_date) AS period_start, MAX(end_date)
AS period_end
FROM grouped
GROUP BY user_id, island_id;
```

Q342 Find the single day with the largest week-over-week percentage change in total revenue.

```
WITH daily AS (
  SELECT sale_date, SUM(amount) AS total
  FROM sales
  GROUP BY sale_date
),
wow AS (
  SELECT sale_date, total,
         LAG(total, 7) OVER (ORDER BY sale_date) AS total_last_week
  FROM daily
)
SELECT sale_date, total, total_last_week,
       ROUND(100.0 * (total - total_last_week) / NULLIF(total_last_week,
0), 2) AS pct_change
FROM wow
ORDER BY ABS(pct_change) DESC
LIMIT 1;
```

Q343 Design a query to detect potential duplicate customer accounts using fuzzy matching on name and phone.

```
SELECT c1.customer_id AS customer_a, c2.customer_id AS customer_b
FROM customers c1
JOIN customers c2
  ON c1.customer_id < c2.customer_id
 AND (
   SOUNDEX(c1.name) = SOUNDEX(c2.name)
   OR c1.phone = c2.phone
 )
WHERE c1.email <> c2.email;
```

16 Senior-Level Interview Challenges

CONTINUED · Q344–Q347

Q344 Build a query that identifies employees eligible for a 'long service' award: 5+ years tenure, no more than 1 manager change, and above-median department performance.

```
WITH tenure AS (
  SELECT id, department_id,
         EXTRACT(YEAR FROM AGE(CURRENT_DATE, join_date)) AS years_service
  FROM employees
),
mgr_changes AS (
  SELECT employee_id, COUNT(*) AS changes
  FROM manager_change_log
  GROUP BY employee_id
),
dept_perf AS (
  SELECT department_id, PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY
performance_score) AS median_score
  FROM performance_reviews pr
  JOIN employees e ON pr.employee_id = e.id
  GROUP BY department_id
)
SELECT e.id, e.name
FROM employees e
JOIN tenure t ON e.id = t.id
LEFT JOIN mgr_changes m ON e.id = m.employee_id
JOIN dept_perf dp ON e.department_id = dp.department_id
JOIN performance_reviews pr ON pr.employee_id = e.id
WHERE t.years_service >= 5
  AND COALESCE(m.changes, 0) <= 1
  AND pr.performance_score > dp.median_score;
```

Q345 Find the shortest path (fewest hops) between two products based on a 'frequently bought together' graph, using recursive CTEs.

```
WITH RECURSIVE graph AS (
  SELECT product_a AS start_product, product_b AS current_product,
ARRAY[product_a, product_b] AS path, 1 AS hops
  FROM product_pairs
  WHERE product_a = 'P100'
  UNION ALL
  SELECT g.start_product, pp.product_b, g.path || pp.product_b, g.hops + 1
  FROM product_pairs pp
  JOIN graph g ON pp.product_a = g.current_product
  WHERE NOT pp.product_b = ANY(g.path) AND g.hops < 5
)
SELECT * FROM graph WHERE current_product = 'P250' ORDER BY hops LIMIT 1;
```

Q346 Compute a complete customer lifetime value (CLV) forecast combining historical average order value, purchase frequency, and estimated retention.

```
WITH customer_stats AS (
  SELECT customer_id,
         AVG(amount) AS avg_order_value,
         COUNT(*)::NUMERIC / NULLIF(EXTRACT(YEAR FROM AGE(MAX(order_date),
MIN(order_date))) + 1, 0) AS orders_per_year
  FROM orders
  GROUP BY customer_id
)
SELECT customer_id, avg_order_value, orders_per_year,
       ROUND(avg_order_value * orders_per_year * 3, 2) AS
estimated_3yr_clv
FROM customer_stats
ORDER BY estimated_3yr_clv DESC;
```

Q347 Write a single query to reconcile two ledgers (system A and system B) and report unmatched transactions on both sides.

```
SELECT 'missing_in_b' AS discrepancy, a.transaction_id, a.amount
FROM ledger_a a
WHERE NOT EXISTS (SELECT 1 FROM ledger_b b WHERE b.transaction_id =
a.transaction_id AND b.amount = a.amount)
UNION ALL
SELECT 'missing_in_a' AS discrepancy, b.transaction_id, b.amount
FROM ledger_b b
WHERE NOT EXISTS (SELECT 1 FROM ledger_a a WHERE a.transaction_id =
b.transaction_id AND a.amount = b.amount);
```

16 Senior-Level Interview Challenges

CONTINUED · Q348–Q350

Q348 Design a query that flags potential salary-band inversions across an entire multi-level reporting chain.

```
WITH RECURSIVE chain AS (
  SELECT id, name, manager_id, salary
  FROM employees
  WHERE manager_id IS NULL
  UNION ALL
  SELECT e.id, e.name, e.manager_id, e.salary
  FROM employees e
  JOIN chain c ON e.manager_id = c.id
)
SELECT e.id AS employee_id, e.name AS employee, e.salary AS
employee_salary,
       m.id AS manager_id, m.name AS manager, m.salary AS manager_salary
FROM employees e
JOIN employees m ON e.manager_id = m.id
WHERE e.salary > m.salary;
```

Q349 Build a single report combining top product per category, category revenue share, and month-over-month growth.

```
WITH cat_month AS (
  SELECT p.category_id, DATE_TRUNC('month', o.order_date) AS month,
         SUM(oi.quantity * oi.price) AS revenue
  FROM order_items oi
  JOIN orders o ON oi.order_id = o.order_id
  JOIN products p ON oi.product_id = p.product_id
  GROUP BY p.category_id, DATE_TRUNC('month', o.order_date)
),
growth AS (
  SELECT category_id, month, revenue,
         revenue - LAG(revenue) OVER (PARTITION BY category_id ORDER BY
month) AS mom_growth,
         ROUND(100.0 * revenue / SUM(revenue) OVER (PARTITION BY month),
2) AS pct_share
  FROM cat_month
)
SELECT * FROM growth ORDER BY month, category_id;
```

Q350 Design a single query that detects employees whose salary, title, and department changed simultaneously in a way that looks like a data-entry error rather than a real promotion.

```
WITH changes AS (
  SELECT employee_id, change_date,
         salary - LAG(salary) OVER (PARTITION BY employee_id ORDER BY
change_date) AS salary_delta,
         title <> LAG(title) OVER (PARTITION BY employee_id ORDER BY
change_date) AS title_changed,
         department_id <> LAG(department_id) OVER (PARTITION BY
employee_id ORDER BY change_date) AS dept_changed
  FROM employee_change_log
)
SELECT *
FROM changes
WHERE salary_delta < 0 AND title_changed AND dept_changed;
```